DEVOPS ENGINEERING MASTERY

CLOUD ENGINEERING — GOOGLE CLOUD PLATFORM

Google Cloud Architecture, IAM, VPC, Compute Engine, Cloud Storage, Cloud SQL, GKE, Cloud Run, Serverless, Security & Production Cloud Operations

VOLUME 13 • FIRST EDITION

```
nahid@technova:~$ gcloud compute instances list
✓ NAME              ZONE          STATUS
✓ technova-web-01   asia-south1-a   RUNNING
nahid@technova:~$ gcloud container clusters list
```

GCP • IAM • VPC • Compute Engine • Cloud SQL • GKE • Cloud Run

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

Companion Workbook • 24 Labs • 3 Projects • Capstone

Terraform Lab Workbook

DEVOPS ENGINEERING MASTERY — Volume 13 (Companion Workbook). First Edition, 9 November 2026. Terraform and HCL with AWS, Azure, GCP and IBM Cloud, built around the TechNova MERN example application. This workbook collects all 24 guided labs, 15 assignments, the 3 major projects and the capstone, so you can work through them beside your terminal without carrying the whole book. Do the labs in order; each lab states its prerequisites. Early labs use local providers so they cost nothing; cloud labs always warn about charges and end with a `terraform destroy` cleanup step. Never run a lab step on production infrastructure. The instructor-style solution guide for the capstone is at the end — attempt the capstone before reading it.

Lab Workbook • Volume 13 • Google Cloud Platform

Lab Workbook — হাতে-কলমে Google Cloud Platform

এই ওয়ার্কবুকটি **DEVOPS ENGINEERING MASTERY — Volume 13: Cloud Engineering — Google Cloud Platform (GCP)** বইয়ের ব্যবহারিক অংশ। বইয়ের প্রতিটি অধ্যায়ে যে ধারণা শেখা হয়েছে, এখানে তা হাতে-কলমে প্রয়োগ করা হবে। প্রতিটি ল্যাব একটি ছোট, সম্পূর্ণ কাজ — শুরু থেকে শেষ পর্যন্ত, যাচাই ও cleanup সহ।

কাল্পনিক কোম্পানি **TechNova** (ডোমেইন `technova.example`, উদাহরণ IP `203.0.113.x`) এই ওয়ার্কবুকের প্রসঙ্গ। সব উদাহরণে TechNova-র প্রজেক্ট, নেটওয়ার্ক ও অ্যাপ্লিকেশন ব্যবহার করা হয়েছে, যাতে ল্যাবগুলো একে অপরের সঙ্গে যুক্ত থাকে এবং শেষে একটি সম্পূর্ণ production-ধরনের প্ল্যাটফর্ম তৈরি হয়।

এই ওয়ার্কবুক কীভাবে ব্যবহার করবেন

- **ক্রমানুসারে করুন:** Lab 1–24 ধারাবাহিকভাবে সাজানো। পরের ল্যাব আগের ল্যাবের উপর নির্ভর করে। কোনো ল্যাব এড়িয়ে গেলে পরের ল্যাবের prerequisite পূরণ হবে না।
- **নিজের প্রজেক্টে করুন:** প্রতিটি ল্যাবে নিজের GCP প্রজেক্ট ব্যবহার করুন। অন্যের প্রজেক্টে বা production প্রজেক্টে ল্যাব করবেন না।
- **খরচের দিকে নজর দিন:** প্রতিটি ল্যাবে cost-risk level দেওয়া আছে — Low, Medium বা High। Lab 1-এ billing budget সেট না করে কোনো billable ল্যাব শুরু করবেন না। দাম কখনো এই বইয়ে hardcode করা হয়নি; সর্বদা **Google Cloud Pricing Calculator** দেখুন।
- **Cleanup বাধ্যতামূলক:** প্রতিটি ল্যাবের শেষে cleanup ধাপ আছে। অপ্রয়োজনীয় billable resource চালু রেখে যাবেন না। ল্যাব শেষ হলে resource মুছে ফেলুন, অথবা পরের ল্যাবে ব্যবহার করবেন বলে নোট করে রাখুন।
- **Least privilege:** কোথাও broad permission (যেমন Owner বা Editor) দেওয়া হয়নি। প্রতিটি ল্যাবে শুধু প্রয়োজনীয় role ব্যবহার করুন। Secret কখনো code-এ, terminal history-তে বা screenshot-এ রাখবেন না — Secret Manager ব্যবহার করুন।

সাধারণ prerequisite

- একটি Google Cloud account ও একটি পরীক্ষামূলক GCP প্রজেক্ট (Lab 1-এ তৈরি করবেন)।
- নিজের কম্পিউটারে `gcloud` CLI ইনস্টল করা (Lab 3-এ বিস্তারিত)।

- মৌলিক Linux terminal, Git ও Node.js জ্ঞান।
- এই বইয়ের Part I–IV-এর ধারণা — IAM, VPC, Compute Engine, Cloud Storage, Cloud Run, GKE, Terraform ও observability।

> **সতর্কতা — খরচ:** GCP-র বেশিরভাগ resource billable। Free tier-এর সীমা ছাড়িয়ে গেলে চার্জ হতে পারে। প্রতিটি ল্যাব শুরু করার আগে billing budget ও alert যাচাই করুন। সন্দেহ হলে resource তৈরি করার আগে Pricing Calculator-এ অনুমান দেখুন।

> **নামকরণের নিয়ম:** এই ওয়ার্কবুকে resource-এর নামে `technova-` prefix ব্যবহার করা হয়েছে, যেমন `technova-vpc`, `technova-web`। নিজের ল্যাবে একই নিয়মে নাম দিন, যাতে কোন resource কোন ল্যাবের তা সহজে বোঝা যায় এবং cleanup-এ ভুল না হয়।

ল্যাবের গঠন

প্রতিটি ল্যাবে একই কাঠামো আছে: Learning Objectives, Prerequisites, Required Services, Estimated Duration, Cost-Risk Level, Console Instructions, gcloud Commands, Expected Output, Verification, Common Errors, Troubleshooting, Security Considerations, Cleanup and Deletion, Reflection Questions। এই কাঠামো অনুসরণ করলে প্রতিটি ল্যাব একটি সম্পূর্ণ, যাচাইযোগ্য কাজ হয়।

Lab	বিষয়	Cost-Risk	আনুমানিক সময়
Lab 1–3	Organization, IAM, gcloud CLI	Low	৩০–৪৫ মিনিট করে
Lab 4–6	VPC, Firewall, Private Access	Low	৪৫–৬০ মিনিট করে
Lab 7–12	Compute, Storage, Load Balancing, DNS/CDN	Medium	৪৫–৯০ মিনিট করে
Lab 13–19	Database, Cloud Run, Pub/Sub, GKE, API	Medium	৬০–৯০ মিনিট করে
Lab 20–24	Terraform, CI/CD, Monitoring, Audit, Production MERN	Medium–High	৬০–১২০ মিনিট করে

Assignments (Assignment 1–15) ল্যাবের জ্ঞানকে নকশা ও বিশ্লেষণের কাজে প্রয়োগ করে। Projects (Project 01–03) ও Capstone-এ সবকিছু একত্র করে একটি সম্পূর্ণ প্ল্যাটফর্ম তৈরি করতে হয়। Capstone Solution বিভাগে reference solution-এর রূপরেখা দেওয়া আছে — প্রথমে নিজে চেষ্টা করুন, তারপর মিলিয়ে দেখুন।

ল্যাব 1 • Lab Workbook

Lab 1 — Organization/Project Setup and Billing Budgets

Learning Objectives

- GCP resource hierarchy — Organization, Folder, Project — ব্যাখ্যা করতে পারবেন।
- নতুন GCP প্রজেক্ট তৈরি, নামকরণ ও লেবেল দিতে পারবেন।
- Billing account যুক্ত করে billing budget ও alert তৈরি করতে পারবেন।
- প্রজেক্টে প্রয়োজনীয় API enable করতে পারবেন।

Prerequisites

- একটি Google account ও একটি billing account (নিজের নামে)।
- Google Cloud Console-এ প্রবেশাধিকার।
- এই বইয়ের Organization ও billing অধ্যায় পড়া।

Required Services

- Cloud Resource Manager, Cloud Billing, Service Usage API।

Estimated Duration

৩০–৪৫ মিনিট।

Cost-Risk Level

Low — এই ল্যাবে কোনো billable resource তৈরি করা হয় না; শুধু প্রজেক্ট, budget ও alert সেট করা হয়।

Console Instructions

1. Google Cloud Console-এ যান। উপরের প্রজেক্ট selector থেকে **New Project** ক্লিক করুন।
2. Project name দিন: `technova-lab`। Organization (যদি থাকে) নির্বাচন করুন; না থাকলে No organization-এ তৈরি করুন।
3. Project ID নোট করুন — এটি unique হতে হবে; প্রয়োজনে Console-এর প্রস্তাবিত ID ব্যবহার করুন।
4. **Billing** → **Budgets & alerts** → **Create budget**-এ যান। Scope-এ `technova-lab` প্রজেক্ট নির্বাচন করুন।
5. Budget amount সেট করুন (উদাহরণ: মাসিক সীমা নিজের সামর্থ্য অনুযায়ী) এবং alert threshold 50%, 90% ও 100%-এ সেট করুন। Email notification চালু করুন।
6. **APIs & Services** → **Library** থেকে Compute Engine API, Cloud Resource Manager API ও Cloud Billing API enable করুন।

7. প্রজেক্টে label যোগ করুন: `env=lab`, `owner=CHANGE-ME-YOUR-NAME`, `cost-center=technova-training`।

gcloud Commands

```
gcloud projects create technova-lab --name="TechNova Lab" --labels=env=lab,cost-
center=technova-training
gcloud config set project technova-lab
gcloud projects describe technova-lab
gcloud services enable compute.googleapis.com cloudresourcemanager.googleapis.com
cloudbilling.googleapis.com
gcloud billing budgets create --billing-account=CHANGE-ME-BILLING-ACCOUNT-ID --display-
name="technova-lab-monthly" --budget-amount=CHANGE-ME-AMOUNT --threshold-rule=percent=50 --
threshold-rule=percent=90 --threshold-rule=percent=100
gcloud billing budgets list --billing-account=CHANGE-ME-BILLING-ACCOUNT-ID
```

> **মনে রাখুন:** Billing account ID জানতে `gcloud billing accounts list` চালান। Budget তৈরি করলে খরচ থেমে যায় না — budget শুধু alert দেয়। খরচ থামাতে resource মুছতে হয়।

Expected Output

```
# Illustrative output — প্রকৃত output আপনার অ্যাকাউন্ট অনুযায়ী ভিন্ন হতে পারে
projectId: technova-lab
name: TechNova Lab
lifecycleState: ACTIVE
# Budget list-এ technova-lab-monthly দেখা যাবে, threshold 50 / 90 / 100 সহ।
```

Verification

- Console-এ প্রজেক্টটি ACTIVE দেখাচ্ছে কি না দেখুন।
- `gcloud config get-value project` চালিয়ে active project `technova-lab` কি না যাচাই করুন।
- Billing → Budgets & alerts-এ budget-টি listed আছে কি না দেখুন।
- `gcloud services list --enabled`-এ Compute Engine API দেখা যাচ্ছে কি না যাচাই করুন।

Common Errors

Error	কারণ	সমাধান
Project ID already exists	ID globally unique নয়	অন্য ID বেছে নিন, যেমন `technova-lab-CHANGE-ME-INITIALS`
Permission denied on billing account	Billing Account User role নেই	Billing admin-কে role দিতে বলুন; নিজের অ্যাকাউন্ট ব্যবহার করুন
API not enabled	Service enable করা হয়নি	`gcloud services enable` আবার চালান

Troubleshooting

- Console-এ প্রজেক্ট না দেখালে পেজ refresh করুন ও সঠিক account-এ login আছেন কি না দেখুন।
- Budget তৈরি না হলে billing account-টি active কি না এবং আপনার role যাচাই করুন।
- `gcloud auth login` দিয়ে পুনরায় authenticate করুন যদি `PERMISSION_DENIED` আসে।

Security Considerations

- প্রজেক্টে অপ্রয়োজনীয় কাউকে Owner role দেবেন না।
- Billing account ID ও প্রজেক্ট ID public জায়গায় (যেমন GitHub) প্রকাশ করবেন না।
- Label-এ কোনো secret, password বা ব্যক্তিগত তথ্য লিখবেন না।

Cleanup and Deletion

- এই ল্যাবে তৈরি প্রজেক্টই পরের ল্যাবগুলোর ভিত্তি — এখন মুছবেন না।
- যদি ভুল প্রজেক্ট তৈরি হয়ে থাকে, `gcloud projects delete CHANGE-ME-WRONG-PROJECT-ID` চালান (৩০ দিনের মধ্যে restore করা যায়)।
- Budget-টি রেখে দিন — এটি পরের সব ল্যাবের খরচ পাহারা দেবে।

Reflection Questions

1. Organization → Folder → Project hierarchy কেন গুরুত্বপূর্ণ? একটি কোম্পানিতে এটি কীভাবে খরচ ও নিরাপত্তা নিয়ন্ত্রণে সাহায্য করে?
2. Billing budget alert পেলে আপনার প্রথম তিনটি পদক্ষেপ কী হবে?
3. প্রজেক্টের নামকরণ ও লেবেলিং কেন operational কাজে সাহায্য করে?

ল্যাব 2 • Lab Workbook

Lab 2 — IAM Roles, Policies and Service Accounts

Learning Objectives

- Least privilege নীতি অনুযায়ী IAM role বরাদ্দ করতে পারবেন।
- Service account তৈরি করে তাতে শুধু প্রয়োজনীয় role দিতে পারবেন।
- IAM policy যাচাই ও audit করতে পারবেন।
- Primitive role (Owner/Editor/Viewer) বনাম predefined role-এর পার্থক্য ব্যাখ্যা করতে পারবেন।

Prerequisites

- Lab 1 সম্পন্ন — `technova-lab` প্রজেক্ট সক্রিয়।
- প্রজেক্টে IAM Admin বা Owner-level প্রবেশাধিকার (নিজের প্রজেক্টে)।

Required Services

- Identity and Access Management (IAM), Cloud Resource Manager।

Estimated Duration

৪৫ মিনিট।

Cost-Risk Level

Low — IAM পরিবর্তনে সরাসরি চার্জ হয় না।

Console Instructions

1. Console → **IAM & Admin** → **IAM**-এ যান। বর্তমান principal-দের তালিকা দেখুন।
2. **Grant Access** ক্লিক করুন। একজন সহকর্মীর (পরীক্ষার জন্য নিজের দ্বিতীয় email বা `CHANGE-ME-TEAMMATE@example.com`) email দিন, role হিসেবে `Compute Viewer` নির্বাচন করুন। Owner/Editor দেবেন না।
3. **IAM & Admin** → **Service Accounts** → **Create Service Account**-এ যান। নাম দিন `technova-app-sa`, description-এ উদ্দেশ্য লিখুন।
4. Service account-টিকে role দিন: `Cloud SQL Client` ও `Storage Object Viewer` — শুধু যা ল্যাবে লাগবে।
5. Service account-এর key তৈরি করবেন না — এই ল্যাবে keyless ব্যবহার (Workload Identity / attached service account) শেখাই লক্ষ্য।
6. **IAM & Admin** → **Audit Logs** চালু আছে কি না যাচাই করুন।

gcloud Commands

```
gcloud iam service-accounts create technova-app-sa --display-name="TechNova App Service
Account" --description="Used by Cloud Run and Compute Engine labs"
gcloud projects add-iam-policy-binding technova-lab --member="serviceAccount:technova-app-
sa@technova-lab.iam.gserviceaccount.com" --role="roles/cloudsql.client"
gcloud projects add-iam-policy-binding technova-lab --member="serviceAccount:technova-app-
sa@technova-lab.iam.gserviceaccount.com" --role="roles/storage.objectViewer"
gcloud projects get-iam-policy technova-lab --format=json
gcloud iam service-accounts list --project=technova-lab
```

সতর্কতা: Service account key (.json) ডাউনলোড করা নিরাপত্তা ঝুঁকি — key চুরি হলে যে কেউ সেই permission ব্যবহার করতে পারে। সম্ভব হলে key তৈরি করবেন না; GCP resource-এর সঙ্গে service account attach করুন।

Expected Output

```
# Illustrative output
# get-iam-policy-তে technova-app-sa-র জন্য roles/cloudsql.client ও
# roles/storage.objectViewer binding দেখা যাবে।
```

Verification

- `gcloud projects get-iam-policy`-এ নতুন binding দুটি আছে কি না দেখুন।
- Console-এ service account-টি listed এবং role দুটি দেখা যাচ্ছে কি না যাচাই করুন।
- কোনো principal-কে Owner/Editor দেওয়া হয়নি তা নিশ্চিত করুন।

Common Errors

Error	কারণ	সমাধান
Permission denied adding binding	IAM Admin role নেই	প্রজেক্ট Owner-কে বলুন বা নিজের প্রজেক্ট ব্যবহার করুন
Service account already exists	আগে তৈরি করা হয়েছে	List করে বিদ্যমান account ব্যবহার করুন
Invalid member format	email ভুল লেখা	`serviceAccount:NAME@PROJECT.iam.gserviceaccount.com` ফরম্যাট যাচাই করুন

Troubleshooting

- Policy পরিবর্তন কার্যকর হতে কয়েক মিনিট লাগতে পারে — অপেক্ষা করে আবার যাচাই করুন।
- Role-এর নাম ভুল হলে `gcloud iam roles list` দিয়ে সঠিক role ID খুঁজুন।

Security Considerations

- Least privilege: শুধু প্রয়োজনীয় role দিন, কখনো `roles/owner` বা `roles/editor` service account-কে দেবেন না।
- Service account key তৈরি করলে তা Secret Manager-এ রাখুন, Git-এ commit করবেন না।
- নিয়মিত `get-iam-policy` দিয়ে অপ্রয়োজনীয় binding আছে কি না audit করুন।

Cleanup and Deletion

- পরীক্ষামূলক teammate binding মুছে ফেলুন: `gcloud projects remove-iam-policy-binding technova-lab --member="user:CHANGE-ME-TEAMMATE@example.com" --role="roles/compute.viewer"`।
- `technova-app-sa` পরের ল্যাবগুলোতে (Lab 13, 15) ব্যবহার হবে — এখন মুছবেন না।

Reflection Questions

1. একটি web অ্যাপের service account-এর কোন কোন role লাগে, আর কোনগুলো লাগে না — কেন?
2. Service account key ব্যবহারের বদলে attached service account কেন নিরাপদ?
3. কোনো কর্মী কোম্পানি ছাড়লে IAM-এ কী কী করতে হয়?

ল্যাব 3 • Lab Workbook

Lab 3 — gcloud CLI Configurations and Authentication

Learning Objectives

- `gcloud` CLI ইনস্টল, authenticate ও configure করতে পারবেন।
- Configuration (account, project, region, zone) পরিচালনা করতে পারবেন।
- Application Default Credentials (ADC) ও `gcloud auth` এর পার্থক্য বুঝতে পারবেন।

Prerequisites

- Lab 1–2 সম্পন্ন। নিজের কম্পিউটারে terminal প্রবেশাধিকার।

Required Services

- Cloud SDK (gcloud CLI)।

Estimated Duration

৩০ মিনিট।

Cost-Risk Level

Low — CLI configuration-এ চার্জ হয় না।

Console Instructions

1. Console-এর উপরে Cloud Shell আইকন (terminal) ক্লিক করুন — ব্রাউজারেই একটি প্রস্তুত terminal পাবেন।
2. Cloud Shell-এ `gcloud config list` চালিয়ে বর্তমান configuration দেখুন।
3. নিজের কম্পিউটারে gcloud ইনস্টল করতে চাইলে অফিসিয়াল Google Cloud SDK ইনস্টল নির্দেশনা অনুসরণ করুন।

gcloud Commands

```
gcloud --version
gcloud auth login
gcloud auth list
gcloud config set project technova-lab
gcloud config set compute/region asia-south1
gcloud config set compute/zone asia-south1-a
gcloud config list
gcloud configurations create technova-lab-config
gcloud configurations activate technova-lab-config
```

```
gcloud auth application-default login
gcloud projects list
```

> **Region ও Zone:** এই ওয়ার্কবুকে `asia-south1` (Mumbai) region ব্যবহার করা হয়েছে, কারণ এটি দক্ষিণ এশিয়ার কাছাকাছি। আপনার ক্ষেত্রে `asia-southeast1` বা অন্য region বেছে নিতে পারেন — তবে সব ল্যাবে একই region ব্যবহার করুন, নইলে resource খুঁজে পেতে সমস্যা হবে।

Expected Output

```
# Illustrative output
[core]
account = CHANGE-ME-YOU@example.com
project = technova-lab
[compute]
region = asia-south1
zone = asia-south1-a
```

Verification

- `gcloud auth list`-এ আপনার account ACTIVE দেখাচ্ছে কি না দেখুন।
- `gcloud config get-value project` = `technova-lab` কি না যাচাই করুন।
- `gcloud compute instances list` কোনো error ছাড়া চলছে কি না দেখুন (খালি তালিকা ঠিক আছে)।

Common Errors

Error	কারণ	সমাধান
command not found: gcloud	SDK ইনস্টল হয়নি বা PATH-এ নেই	অফিসিয়াল ইনস্টলার দিয়ে ইনস্টল করুন; Cloud Shell ব্যবহার করুন
Re-authentication required	Token expired	`gcloud auth login` আবার চালান
Quota / project not set	Active project সেট হয়নি	`gcloud config set project technova-lab` চালান

Troubleshooting

- একাধিক account থাকলে `gcloud config configurations list` দিয়ে কোন configuration active তা দেখুন।
- ADC সমস্যায় `gcloud auth application-default login` পুনরায় চালান।
- Cloud Shell-এ কাজ করলে নিজের কম্পিউটারে কিছু ইনস্টল করতে হয় না।

Security Considerations

- নিজের credential অন্যের কম্পিউটারে `gcloud auth login` করে রাখবেন না; কাজ শেষে `gcloud auth revoke` করুন।
- ADC file (`application_default_credentials.json`) কাউকে দেবেন না, Git-এ তুলবেন না।

Cleanup and Deletion

- এই ল্যাবে কোনো billable resource তৈরি হয়নি — মুছতে কিছু নেই।
- অতিরিক্ত test configuration থাকলে `gcloud configurations delete CHANGE-ME-CONFIG-NAME` দিয়ে মুছুন।

Reflection Questions

1. `gcloud auth login` ও ADC-র ব্যবহার কখন আলাদা?
2. একাধিক প্রজেক্টে কাজ করলে configuration আলাদা রাখা কেন জরুরি?
3. CI/CD-তে মানুষের credential ব্যবহার না করে কী ব্যবহার করা উচিত?

ল্যাব 4 • Lab Workbook

Lab 4 — Custom VPC and Subnet Design

Learning Objectives

- Default VPC বনাম Custom VPC-র পার্থক্য ব্যাখ্যা করতে পারবেন।
- Custom VPC, regional subnet ও secondary IP range তৈরি করতে পারবেন।
- IP CIDR পরিকল্পনা করে subnet-এ IP বরাদ্দ করতে পারবেন।

Prerequisites

- Lab 1–3 সম্পন্ন। VPC ও CIDR ধারণা পড়া।

Required Services

- Compute Engine (VPC networking), Cloud Resource Manager।

Estimated Duration

৪৫–৬০ মিনিট।

Cost-Risk Level

Low — VPC ও subnet নিজে billable নয় (তবে এর ভেতরের VM, NAT ইত্যাদি billable)।

Console Instructions

1. Console → **VPC network** → **VPC networks** → **Create VPC network**।
2. Name: `technova-vpc`। Subnet creation mode: **Custom**।
3. প্রথম subnet: নাম `technova-web-subnet`, region `asia-south1`, IPv4 range `10.10.1.0/24`। Private Google Access: On।
4. **Add subnet** করে দ্বিতীয় subnet: `technova-db-subnet`, region `asia-south1`, range `10.10.2.0/24`, Private Google Access: On।
5. Firewall rules: এই ধাপে default rule রাখুন; Lab 5-এ বিস্তারিত নিয়ম বানাবেন। **Create** ক্লিক করুন।
6. VPC network-এর বিবরণে দুটি subnet ও তাদের CIDR দেখুন।

Internet → technova-vpc (10.10.0.0/16) → technova-web-subnet 10.10.1.0/24 (asia-south1) → technova-db-subnet 10.10.2.0/24 (asia-south1) → Private Google Access → Google APIs

gcloud Commands

```
gcloud compute networks create technova-vpc --subnet-mode=custom --bgp-routing-mode=regional
gcloud compute networks subnets create technova-web-subnet --network=technova-vpc --
region=asia-south1 --range=10.10.1.0/24 --enable-private-ip-google-access
gcloud compute networks subnets create technova-db-subnet --network=technova-vpc --
region=asia-south1 --range=10.10.2.0/24 --enable-private-ip-google-access
gcloud compute networks describe technova-vpc
gcloud compute networks subnets list --network=technova-vpc --regions=asia-south1
```

Expected Output

```
# Illustrative output
# subnets list-এ technova-web-subnet (10.10.1.0/24) ও
# technova-db-subnet (10.10.2.0/24) দেখা যাবে, privateIpGoogleAccess: true সহ।
```

Verification

- Console-এ VPC-তে দুটি subnet দেখা যাচ্ছে কি না দেখুন।
- `gcloud compute networks subnets describe technova-web-subnet --region=asia-south1`-এ CIDR ও Private Google Access যাচাই করুন।
- Default VPC-র সঙ্গে এই VPC-র নাম গুলিয়ে ফেলেননি তা নিশ্চিত করুন।

Common Errors

Error	কারণ	সমাধান
IP range overlaps	CIDR অন্য subnet-এর সঙ্গে মিলে গেছে	আলাদা, non-overlapping CIDR বেছে নিন
Invalid CIDR	ভুল range লেখা	`10.10.1.0/24` ফরম্যাট যাচাই করুন
Network already exists	আগে তৈরি করা হয়েছে	List করে বিদ্যমান VPC ব্যবহার করুন

Troubleshooting

- Subnet তৈরি না হলে region সঠিক কি না দেখুন — subnet regional resource।
- IP পরিকল্পনার জন্য একটি ছোট টেবিল আগে থেকে লিখে নিন (কোন subnet, কোন CIDR, কোন কাজ)।

Security Considerations

- Database subnet-এ public IP দেবেন না — শুধু private IP ব্যবহার করুন।
- Private Google Access চালু রাখুন, যাতে private VM Google API ব্যবহার করতে পারে।
- Default VPC ব্যবহার করবেন না — তার firewall নিয়ম অনেক বেশি খোলা।

Cleanup and Deletion

- `technova-vpc` পরের ল্যাবগুলোর ভিত্তি — এখন মুছবেন না।
- ভুল subnet মুছতে: `gcloud compute networks subnets delete CHANGE-ME-SUBNET --region=asia-south1` (subnet-এ resource থাকলে আগে সেগুলো মুছুন)।

Reflection Questions

1. Web ও database-এর জন্য আলাদা subnet কেন দরকার?
2. Secondary IP range কখন লাগে (যেমন GKE-র জন্য)?
3. CIDR ছোট রাখলে ভবিষ্যতে কী সমস্যা হতে পারে?

ল্যাব 5 • Lab Workbook

Lab 5 — Firewall Rules, Routes and Cloud NAT

Learning Objectives

- Least-privilege firewall rule লিখতে পারবেন (source, target tag, port)।
- Custom route ও Cloud NAT তৈরি করে private VM-কে outbound internet দিতে পারবেন।
- Firewall rule-এর priority ও deny নীতি ব্যাখ্যা করতে পারবেন।

Prerequisites

- Lab 4 সম্পন্ন — `technova-vpc` ও subnet দুটি তৈরি।

Required Services

- Compute Engine, Cloud NAT, Cloud Router।

Estimated Duration

৬০ মিনিট।

Cost-Risk Level

Medium — Cloud NAT gateway ও Cloud Router billable; ল্যাব শেষে মুছে ফেলুন।

Console Instructions

1. Console → **VPC network** → **Firewall** → **Create Firewall Rule**।
2. Rule 1 — SSH (নিজের IP থেকে): নাম `technova-allow-ssh-my-ip`, network `technova-vpc`, direction Ingress, action Allow, targets: Specified target tags `technova-web`, source IPv4 ranges: আপনার public IP `203.0.113.10/32` (উদাহরণ; নিজের আসল IP বসান), protocol TCP port `22`।
3. Rule 2 — Web: নাম `technova-allow-http-https`, targets `technova-web`, source `0.0.0.0/0`, TCP ports `80,443`।
4. Rule 3 — Internal: নাম `technova-allow-internal`, source ranges `10.10.0.0/16`, protocol TCP/UDP/ICMP সব।
5. **Cloud NAT:** VPC network → Cloud NAT → Get started। Gateway নাম `technova-nat`, VPC `technova-vpc`, region `asia-south1`, Cloud Router: Create new router `technova-router`। NAT mapping: সব subnet-এর সব IP range।
6. Firewall rule তালিকায় তিনটি rule ও তাদের priority (default 1000) দেখুন।

gcloud Commands

```
gcloud compute firewall-rules create technova-allow-ssh-my-ip --network=technova-vpc --
direction=INGRESS --action=ALLOW --target-tags=technova-web --source-ranges=203.0.113.10/32 --
rules=tcp:22 --priority=1000
gcloud compute firewall-rules create technova-allow-http-https --network=technova-vpc --
direction=INGRESS --action=ALLOW --target-tags=technova-web --source-ranges=0.0.0.0/0 --
rules=tcp:80,tcp:443
gcloud compute firewall-rules create technova-allow-internal --network=technova-vpc --
direction=INGRESS --action=ALLOW --source-ranges=10.10.0.0/16 --rules=tcp,udp,icmp
gcloud compute routers create technova-router --network=technova-vpc --region=asia-south1
gcloud compute routers nats create technova-nat --router=technova-router --region=asia-south1
--nat-all-subnet-ip-ranges --auto-allocate-nat-external-ips
gcloud compute firewall-rules list --filter="network:technova-vpc"
```

সতর্কতা — SSH: SSH rule-এ `0.0.0.0/0` দেবেন না। শুধু নিজের IP (`/32`) থেকে SSH খুলুন। প্রয়োজনে IAP (Identity-Aware Proxy) ব্যবহার করুন — এটি Lab 23-এর security review-এ আবার আসবে।

Expected Output

```
# Illustrative output
# firewall-rules list-এ তিনটি rule দেখা যাবে:
# technova-allow-ssh-my-ip  INGRESS  ALLOW  tcp:22  203.0.113.10/32
# technova-allow-http-https INGRESS  ALLOW  tcp:80,443  0.0.0.0/0
# technova-allow-internal   INGRESS  ALLOW  tcp,udp,icmp  10.10.0.0/16
```

Verification

- `gcloud compute firewall-rules list`-এ তিনটি rule আছে কি না দেখুন।
- Cloud NAT status: `gcloud compute routers nats describe technova-nat --router=technova-router --region=asia-south1`।
- Lab 7-এ VM তৈরি করে NAT দিয়ে outbound internet (যেমন `apt update`) কাজ করে কি না পরীক্ষা করবেন।

Common Errors

Error	কারণ	সমাধান
SSH timeout	Firewall tag মেলেনি বা source IP ভুল	VM-এর tag `technova-web` কি না ও নিজের IP যাচাই করুন
NAT not working	Router/NAT region ভুল	NAT ও subnet একই region-এ আছে কি না দেখুন
Rule conflict	একই priority-তে deny rule	Priority নম্বর যাচাই করুন — ছোট নম্বর আগে কার্যকর হয়

Troubleshooting

- Connectivity Test (Network Intelligence Center) দিয়ে VM থেকে internet-এর পথ পরীক্ষা করুন।

- `gcloud compute firewall-rules describe` দিয়ে rule-এর source/target যাচাই করুন।

Security Considerations

- SSH শুধু নিজের IP থেকে; RDP/SSH কখনো `0.0.0.0/0` থেকে খুলবেন না।
- HTTP (80) শুধু redirect-এর জন্য; production-এ HTTPS (443) বাধ্যতামূলক।
- প্রতিটি rule-এ description লিখুন — কে, কেন, কবে খুলেছে তা পরে audit-এ লাগে।

Cleanup and Deletion

- Firewall rule ও NAT পরের ল্যাবগুলোতে লাগবে — এখন মুছবেন না।
- ল্যাব শেষে NAT মুছতে চাইলে: `gcloud compute routers nats delete technova-nat --router=technova-router --region=asia-south1` এবং `gcloud compute routers delete technova-router --region=asia-south1`।

Reflection Questions

1. Cloud NAT ছাড়া private VM কীভাবে software update করবে?
2. Firewall rule-এ tag ব্যবহারের সুবিধা কী? IP-based rule-এর তুলনায়?
3. একটি deny rule কখন allow rule-কে override করে?

ল্যাব 6 • Lab Workbook

Lab 6 — Private Google Access and Private Service Connect

Learning Objectives

- Private Google Access (PGA) কী ও কেন লাগে তা ব্যাখ্যা করতে পারবেন।
- Private Service Connect (PSC) দিয়ে Google API ও managed service-এ privateভাবে সংযোগ করতে পারবেন।
- DNS ও routing যাচাই করতে পারবেন।

Prerequisites

- Lab 4–5 সম্পন্ন। Lab 7-এর VM ধারণা জানা (এই ল্যাবে একটি ছোট test VM ব্যবহার করা হবে)।

Required Services

- Compute Engine, Private Service Connect, Cloud DNS।

Estimated Duration

৬০ মিনিট।

Cost-Risk Level

Low–Medium — PSC endpoint ও test VM billable; ল্যাব শেষে মুছুন।

Console Instructions

1. Lab 4-এ `technova-web-subnet`-এ Private Google Access On করা হয়েছে — Console → VPC network → Subnet details-এ যাচাই করুন। না থাকলে Edit করে On করুন।
2. একটি test VM তৈরি করুন (Lab 7-এর ধাপ অনুসরণ করে): নাম `technova-private-test`, subnet `technova-web-subnet`, **External IP: None**, tag `technova-web`।
3. VM-এ SSH করুন (IAP tunnel ব্যবহার করুন: Console → VM → SSH → Open in browser window, অথবা `gcloud compute ssh --tunnel-through-iap`)।
4. VM-এর ভেতর থেকে পরীক্ষা করুন: `curl -I https://storage.googleapis.com` ও `ping -c 3 8.8.8.8` — প্রথমটি PGA দিয়ে কাজ করা উচিত, দ্বিতীয়টি NAT ছাড়া ব্যর্থ হতে পারে (এটাই প্রত্যাশিত পার্থক্য)।
5. **Private Service Connect:** VPC network → Private Service Connect → Create endpoint। Target: Google APIs bundle (`all-apis`), নাম `technova-psc-all-apis`, network `technova-vpc`, subnet `technova-web-subnet`, IP address: একটি private IP (যেমন `10.10.1.100`)।

6. PSC endpoint তৈরি হলে VM থেকে `nslookup storage.googleapis.com` চালিয়ে private IP-তে resolve হচ্ছে কি না দেখুন।

gcloud Commands

```
gcloud compute networks subnets update technova-web-subnet --region=asia-south1 --enable-private-ip-google-access
gcloud compute instances create technova-private-test --zone=asia-south1-a --machine-type=e2-micro --subnet=technova-web-subnet --no-address --tags=technova-web --image-family=debian-12 --image-project=debian-cloud
gcloud compute ssh technova-private-test --zone=asia-south1-a --tunnel-through-iap
gcloud compute addresses create technova-psc-all-apis-ip --region=asia-south1 --subnet=technova-web-subnet --addresses=10.10.1.100 --purpose=PRIVATE_SERVICE_CONNECT
gcloud compute forwarding-rules create technova-psc-all-apis --region=asia-south1 --network=technova-vpc --address=technova-psc-all-apis-ip --target-google-apis-bundle=all-apis
```

Expected Output

```
# Illustrative output — VM-এর ভেতর থেকে:
# $ curl -I https://storage.googleapis.com
# HTTP/2 200
# $ nslookup storage.googleapis.com
# Address: 10.10.1.100  (PSC endpoint IP)
```

Verification

- Private VM থেকে Google API-তে সংযোগ হচ্ছে, কিন্তু public internet-এ সরাসরি ping যাচ্ছে না — এই পার্থক্যটি নোট করুন।
- `gcloud compute forwarding-rules describe technova-psc-all-apis --region=asia-south1`-এ PSC endpoint ACTIVE কি না দেখুন।
- Subnet-এর PGA status On কি না যাচাই করুন।

Common Errors

Error	কারণ	সমাধান
curl timeout to Google APIs	PGA Off বা DNS ভুল	Subnet-এ PGA On করুন; DNS যাচাই করুন
PSC IP already in use	IP subnet-এ ব্যবহৃত	খালি private IP বেছে নিন
IAP SSH fails	IAP firewall rule নেই	Source `35.235.240.0/20` থেকে TCP 22 allow করুন

Troubleshooting

- `dig` / `nslookup` দিয়ে DNS resolution যাচাই করুন — PSC ব্যবহার করলে Google API private IP-তে resolve হয়।

- Cloud NAT বন্ধ থাকলে private VM-এর general internet কাজ করবে না — এটি স্বাভাবিক; PGA শুধু Google API-র জন্য।

Security Considerations

- Private VM-এ public IP না দেওয়াই নিরাপত্তার মূল নীতি — আক্রমণের পথ (attack surface) কমে।
- PSC ব্যবহার করলে ডেটা public internet দিয়ে যায় না।
- IAP SSH ব্যবহার করলে SSH port public-এ খুলতে হয় না।

Cleanup and Deletion

- Test VM মুছুন: `gcloud compute instances delete technova-private-test --zone=asia-south1-a`।
- PSC endpoint মুছুন: `gcloud compute forwarding-rules delete technova-psc-all-apis --region=asia-south1` ও reserved address মুছুন।
- VPC, subnet ও NAT রেখে দিন — পরের ল্যাবে লাগবে।

Reflection Questions

1. Private Google Access ও Cloud NAT-এর কাজের পার্থক্য কী?
2. PSC কখন ব্যবহার করবেন, আর কখন শুধু PGA-ই যথেষ্ট?
3. Database-কে public IP না দিয়ে PSC দিয়ে সংযোগ করলে কী সুবিধা?

ল্যাব 7 • Lab Workbook

Lab 7 — Compute Engine Deployment

Learning Objectives

- Compute Engine VM তৈরি, SSH ও startup script দিয়ে web server চালাতে পারবেন।
- Machine type, image, disk ও network সঠিকভাবে বেছে নিতে পারবেন।
- VM-এর service account attach করতে পারবেন।

Prerequisites

- Lab 2 (service account), Lab 4–5 (VPC, firewall, NAT) সম্পন্ন।

Required Services

- Compute Engine, Cloud Logging (agent ছাড়া basic)।

Estimated Duration

৪৫–৬০ মিনিট।

Cost-Risk Level

Medium — VM চলমান থাকলেই চার্জ হয়; ল্যাব শেষে stop বা delete করুন।

Console Instructions

1. Console → **Compute Engine** → **VM instances** → **Create Instance**।
2. Name: `technova-web-1`, Region `asia-south1`, Zone `asia-south1-a`।
3. Machine type: `e2-micro` (ল্যাবের জন্য যথেষ্ট)। Boot disk: Debian 12, 10 GB Standard persistent disk।
4. Networking: Network `technova-vpc`, Subnetwork `technova-web-subnet`, External IP: Ephemeral। Network tags: `technova-web`।
5. Service account: Lab 2-এর `technova-app-sa` নির্বাচন করুন। Access scopes: **Set access for each API** থেকে শুধু প্রয়োজনীয় scope দিন, অথবা Cloud Platform scope-এর বদলে নির্দিষ্ট scope ব্যবহার করুন।
6. Management → Automation → Startup script-এ নিচের script দিন (একটি সহজ web server):

```
#!/bin/bash
apt-get update
apt-get install -y nginx
systemctl enable nginx
```

```
systemctl start nginx
echo "Hello from TechNova - technova-web-1" > /var/www/html/index.html
```

1. **Create** ক্লিক করুন। VM RUNNING হলে external IP নোট করুন।
2. ব্রাউজারে `http://CHANGE-ME-EXTERNAL-IP` খুলে "Hello from TechNova" দেখুন।

gcloud Commands

```
gcloud compute instances create technova-web-1 --zone=asia-south1-a --machine-type=e2-micro --
subnet=technova-web-subnet --tags=technova-web --image-family=debian-12 --image-
project=debian-cloud --boot-disk-size=10GB --service-account=technova-app-sa@technova-
lab.iam.gserviceaccount.com --metadata-from-file=startup-script=CHANGE-ME-STARTUP-SCRIPT.sh
gcloud compute instances list --filter="name:technova-web-1"
gcloud compute ssh technova-web-1 --zone=asia-south1-a
gcloud compute instances describe technova-web-1 --zone=asia-south1-a --
format="get(networkInterfaces[0].accessConfigs[0].natIP)"
```

Expected Output

```
# Illustrative output
# NAME            ZONE           MACHINE_TYPE  INTERNAL_IP   EXTERNAL_IP     STATUS
# technova-web-1   asia-south1-a  e2-micro      10.10.1.5     203.0.113.20    RUNNING
# ব্রাউজারে: Hello from TechNova - technova-web-1
```

Verification

- VM status RUNNING কি না দেখুন।
- External IP-তে HTTP 200 আসছে কি না: `curl -I http://CHANGE-ME-EXTERNAL-IP`।
- SSH করে `systemctl status nginx` active কি না দেখুন।
- Serial console log-এ startup script-এর output আছে কি না দেখুন।

Common Errors

Error	কারণ	সমাধান
Connection refused on port 80	Firewall tag মেলেনি বা nginx চলছে না	Tag `technova-web` ও Lab 5-এর HTTP rule যাচাই করুন
Quota exceeded	Region-এ VM quota শেষ	অন্য zone বা ছোট machine type ব্যবহার করুন; quota বাড়াতে request করুন
SSH permission denied	OS Login বা key সমস্যা	`gcloud compute ssh` ব্যবহার করুন; IAP tunnel চেষ্টা করুন

Troubleshooting

- Startup script কাজ না করলে VM-এর serial port output দেখুন: Console → VM → Logs → Serial port 1।
- `sudo journalctl -u nginx` দিয়ে nginx-এর log দেখুন।

Security Considerations

- VM-এ service account-এর শুধু প্রয়োজনীয় permission দিন; default Compute Engine service account-এর broad scope এড়িয়ে চলুন।
- OS patch নিয়মিত করুন; unattended upgrade বিবেচনা করুন।
- SSH key নিজের কাছে রাখুন; password authentication বন্ধ রাখুন।

Cleanup and Deletion

- ল্যাব শেষে VM মুছুন (পরের ল্যাব Lab 10-এ আবার instance template থেকে তৈরি করবেন): `gcloud compute instances delete technova-web-1 --zone=asia-south1-a`।
- VM stop করে রাখলেও disk-এর চার্জ চলতে থাকে — সম্পূর্ণ মুছে ফেলাই নিরাপদ, যদি না পরের ল্যাবে একই VM লাগে।

Reflection Questions

1. e2-micro বনাম e2-medium — কখন কোনটি বেছে নেবেন?
2. Startup script-এর বদলে image বা configuration management ব্যবহারের সুবিধা কী?
3. VM-এর external IP ephemeral হলে কী সমস্যা হতে পারে?

ল্যাব 8 • Lab Workbook

Lab 8 — Persistent Disk, Snapshots and Recovery

Learning Objectives

- অতিরিক্ত Persistent Disk attach, format ও mount করতে পারবেন।
- Snapshot তৈরি করে তা থেকে disk ও VM পুনরুদ্ধার (restore) করতে পারবেন।
- Snapshot schedule ও retention নীতি ব্যাখ্যা করতে পারবেন।

Prerequisites

- Lab 7 সম্পন্ন (VM তৈরি ও SSH জানা)।

Required Services

- Compute Engine (Persistent Disk, Snapshots)।

Estimated Duration

৬০ মিনিট।

Cost-Risk Level

Medium — Disk ও snapshot storage billable; ল্যাব শেষে মুছুন।

Console Instructions

1. Lab 7-এর VM আবার তৈরি করুন (নাম `technova-web-1`) অথবা বিদ্যমান VM ব্যবহার করুন।
2. Console → **Compute Engine** → **Storage** → **Disks** → **Create Disk**। নাম `technova-data-disk`, region/zone `asia-south1-a`, size 10 GB, type Standard persistent disk।
3. VM-এর বিবরণে **Edit** → **Additional disks** → **Attach existing disk** থেকে disk-টি attach করুন।
4. VM-এ SSH করে disk format ও mount করুন:

```
lsblk
sudo mkfs.ext4 -m 0 -E lazy_itable_init=0,lazy_journal_init=0,discard /dev/sdb
sudo mkdir -p /mnt/data
sudo mount -o discard,defaults /dev/sdb /mnt/data
echo "TechNova data disk - test file" | sudo tee /mnt/data/hello.txt
ls -l /mnt/data/
```

1. Console → **Snapshots** → **Create Snapshot**। নাম `technova-data-snap-1`, source disk `technova-data-disk`।

2. Recovery পরীক্ষা: নতুন disk তৈরি করুন snapshot থেকে (`technova-data-disk-restored`), একটি test VM-এ attach করে `/mnt/data/hello.txt` আছে কি না দেখুন।
3. Snapshot schedule তৈরি করুন: Snapshots → Snapshot schedules → Create, দৈনিক 1টি snapshot, retention 7 দিন।

gcloud Commands

```
gcloud compute disks create technova-data-disk --zone=asia-south1-a --size=10GB --type=pd-standard
gcloud compute instances attach-disk technova-web-1 --disk=technova-data-disk --zone=asia-south1-a
gcloud compute snapshots create technova-data-snap-1 --source-disk=technova-data-disk --source-disk-zone=asia-south1-a
gcloud compute disks create technova-data-disk-restored --zone=asia-south1-a --source-snapshot=technova-data-snap-1 --size=10GB
gcloud compute snapshots list --filter="name:technova-data-snap-1"
gcloud compute resource-policies create snapshot-schedule technova-daily-snap --region=asia-south1 --max-retention-days=7 --on-source-disk-delete=keep-auto-snapshots --daily-schedule --start-time=02:00
```

Expected Output

```
# Illustrative output — VM-এর ভেতর:
# $ ls -l /mnt/data/
# -rw-r--r-- 1 root root 32 ... hello.txt
# $ cat /mnt/data/hello.txt
# TechNova data disk - test file
# Restored disk-এও একই file পাওয়া যাবে।
```

Verification

- `df -h /mnt/data`-এ disk mounted দেখা যাচ্ছে কি না দেখুন।
- Snapshot status READY কি না: `gcloud compute snapshots describe technova-data-snap-1`।
- Restored disk-এ file-টি অক্ষত আছে কি না যাচাই করুন।

Common Errors

Error	কারণ	সমাধান
Device not found (/dev/sdb)	Disk attach হয়নি বা device নাম ভিন্ন	`lsblk` দিয়ে আসল device নাম দেখুন
Mount fails after reboot	/etc/fstab-এ entry নেই	UUID দিয়ে fstab-এ entry যোগ করুন
Snapshot in progress	আগের snapshot চলছে	শেষ হওয়া পর্যন্ত অপেক্ষা করুন

Troubleshooting

- Disk attach করার পর VM reboot না করলে `lsblk` দিয়ে নতুন device দেখা যায় কি না যাচাই করুন।
- Snapshot থেকে disk তৈরি করার সময় size source disk-এর সমান বা বড় হতে হবে।

Security Considerations

- Disk encryption: Google-managed encryption default; সংবেদনশীল ডেটার জন্য CMEK (Customer-Managed Encryption Key) বিবেচনা করুন।
- Snapshot-এ ডেটা থাকে — snapshot শেয়ার করার আগে ভাবুন; public করবেন না।
- Snapshot schedule-এ retention সীমা দিন, নইলে storage খরচ বাড়বে।

Cleanup and Deletion

- Restored test disk ও test VM মুছুন।
- Snapshot মুছুন: `gcloud compute snapshots delete technova-data-snap-1`।
- Data disk detach করে মুছুন: `gcloud compute instances detach-disk technova-web-1 --disk=technova-data-disk --zone=asia-south1-a`, তারপর `gcloud compute disks delete technova-data-disk --zone=asia-south1-a`।
- VM মুছুন (Lab 10-এ template থেকে আবার তৈরি হবে)।

Reflection Questions

1. Snapshot বনাম image-এর পার্থক্য কী? কখন কোনটি ব্যবহার করবেন?
2. Snapshot schedule-এর retention 7 দিন রাখার সুবিধা ও ঝুঁকি কী?
3. Disk মুছে গেলে snapshot থেকে কী কী পুনরুদ্ধার করা যায়, কী যায় না?

ল্যাব 9 • Lab Workbook

Lab 9 — Cloud Load Balancing

Learning Objectives

- External Application Load Balancer তৈরি করতে পারবেন।
- Instance group, backend service, health check ও frontend configure করতে পারবেন।
- একটি VM বন্ধ হলে traffic অন্য VM-এ যায় কি না পরীক্ষা করতে পারবেন।

Prerequisites

- Lab 7–8 সম্পন্ন। দুটি VM চালানোর ধারণা জানা।

Required Services

- Compute Engine, Cloud Load Balancing, Cloud Armor (ঐচ্ছিক)।

Estimated Duration

৯০ মিনিট।

Cost-Risk Level

Medium — Load balancer, forwarding rule ও backend VM billable; ল্যাব শেষে মুছুন।

Console Instructions

1. দুটি VM তৈরি করুন: `technova-web-1` ও `technova-web-2` (Lab 7-এর ধাপে, প্রতিটিতে nginx ও আলাদা "Hello from technova-web-1 / 2" পেজ)।
2. Console → **Network services** → **Load balancing** → **Create Load Balancer** → **Application Load Balancer (external)** → **Global external Application Load Balancer**।
3. Frontend: নাম `technova-lb-frontend`, protocol HTTP, port 80, IP: Ephemeral।
4. Backend: Backend type: Instance group। নতুন unmanaged instance group তৈরি করুন `technova-web-ig`, zone `asia-south1-a`, দুটি VM যোগ করুন, port 80।
5. Health check: নাম `technova-web-health`, protocol HTTP, port 80, path `/`, check interval 10s, timeout 5s, healthy threshold 2, unhealthy threshold 3।
6. Routing rules: default — সব traffic backend service-এ। **Create** ক্লিক করুন।
7. Load balancer-এর IP নোট করুন। ব্রাউজারে বারবার refresh করে দুটি VM-এর পেজ পর্যায়ক্রমে আসছে কি না দেখুন।
8. একটি VM stop করুন — কিছুক্ষণ পর সব request সুস্থ VM-এ যাচ্ছে কি না পরীক্ষা করুন। তারপর VM আবার start করুন।

gcloud Commands

```
gcloud compute instance-groups unmanaged create technova-web-ig --zone=asia-south1-a
gcloud compute instance-groups unmanaged add-instances technova-web-ig --instances=technova-web-1,technova-web-2 --zone=asia-south1-a
gcloud compute health-checks create http technova-web-health --port=80 --request-path=/ --check-interval=10s --timeout=5s --healthy-threshold=2 --unhealthy-threshold=3
gcloud compute backend-services create technova-web-backend --protocol=HTTP --port-name=http --health-checks=technova-web-health --global
gcloud compute backend-services add-backend technova-web-backend --instance-group=technova-web-ig --instance-group-zone=asia-south1-a --global
gcloud compute url-maps create technova-web-map --default-service=technova-web-backend
gcloud compute target-http-proxies create technova-web-proxy --url-map=technova-web-map
gcloud compute forwarding-rules create technova-lb-frontend --global --target-http-proxy=technova-web-proxy --ports=80
gcloud compute forwarding-rules describe technova-lb-frontend --global --format="get(IPAddress)"
```

Expected Output

```
# Illustrative output
# $ curl http://CHANGE-ME-LB-IP
# Hello from TechNova - technova-web-1
# $ curl http://CHANGE-ME-LB-IP
# Hello from TechNova - technova-web-2
# একটি VM বন্ধ করলে সব response সুস্থ VM থেকে আসবে।
```

Verification

- Load balancer IP-তে বারবার `curl` করে দুটি VM-এর response পাচ্ছেন কি না দেখুন।
- Backend service-এর health: Console → Load balancing → Backend-এ দুটি instance HEALTHY কি না দেখুন।
- একটি VM বন্ধ করে health check UNHEALTHY হয় কি না ও traffic সরে যায় কি না যাচাই করুন।

Common Errors

Error	কারণ	সমাধান
Backend UNHEALTHY	Firewall-এ health check IP range বন্ধ	Source `130.211.0.0/22` ও `35.191.0.0/16` থেকে TCP 80 allow করুন
502 Bad Gateway	Backend VM-এ nginx বন্ধ	VM-এ `systemctl status nginx` দেখুন
LB IP-তে কোনো response নেই	Forwarding rule বা URL map ভুল	Proxy, URL map ও backend service-এর সংযোগ যাচাই করুন

Troubleshooting

- Health check-এর জন্য Google-এর health check IP range-গুলোতে firewall খোলা আছে কি না প্রথমে দেখুন — এটাই সবচেয়ে সাধারণ ভুল।
- Backend service-এর log Cloud Logging-এ দেখুন।

Security Considerations

- Backend VM-এর HTTP port শুধু load balancer ও health check-এর জন্য খুলুন; সরাসরি public access সীমিত করুন।
- Production-এ HTTP-র বদলে HTTPS ব্যবহার করুন (Lab 12)।
- Cloud Armor দিয়ে rate limiting ও WAF নিয়ম বিবেচনা করুন।

Cleanup and Deletion

- Load balancer মুছুন: forwarding rule, target proxy, URL map, backend service, health check — এই ক্রমে মুছুন।
- Instance group ও দুটি VM মুছুন।
- Lab 10-এ Managed Instance Group দিয়ে আবার তৈরি করবেন — তাই VM-গুলো এখন মুছে ফেলুন।

Reflection Questions

1. Health check কেন load balancer-এর প্রাণ? Health check ভুল হলে কী হয়?
2. Global load balancer বনাম regional load balancer — কখন কোনটি?
3. একটি VM বন্ধ হলে ব্যবহারকারী কী অনুভব করবে, যদি load balancer না থাকত?

ল্যাব 10 • Lab Workbook

Lab 10 — Managed Instance Groups and Autoscaling

Learning Objectives

- Instance template ও Managed Instance Group (MIG) তৈরি করতে পারবেন।
- CPU utilization-এর ভিত্তিতে autoscaling configure করতে পারবেন।
- Autohealing ও rolling update পরীক্ষা করতে পারবেন।

Prerequisites

- Lab 7, 9 সম্পন্ন। Load balancer ধারণা জানা।

Required Services

- Compute Engine (MIG, Autoscaler), Cloud Monitoring।

Estimated Duration

৯০ মিনিট।

Cost-Risk Level

Medium–High — Autoscaling-এ VM সংখ্যা বাড়তে পারে; min/max সীমা ছোট রাখুন ও ল্যাব শেষে MIG মুছুন।

Console Instructions

1. Console → **Compute Engine** → **Instance templates** → **Create Instance Template**। নাম `technova-web-template`, machine `e2-micro`, image Debian 12, network `technova-vpc`, subnet `technova-web-subnet`, tags `technova-web`, service account `technova-app-sa`, startup script-এ nginx ইনস্টল (Lab 7-এর script)।
2. **Instance groups** → **Create Instance Group** → **New managed instance group**। নাম `technova-web-mig`, region `asia-south1`, template `technova-web-template`।
3. Autoscaling: **On**। Minimum number of instances: 1, Maximum: 3। Autoscaling metric: CPU utilization, target 60%। Cooldown period: 60s।
4. Autohealing: Health check Lab 9-এর `technova-web-health` ব্যবহার করুন (আবার তৈরি করুন যদি মুছে থাকে)। Initial delay: 300s।
5. **Create** করুন। কয়েক মিনিট অপেক্ষা করুন — 1টি VM তৈরি হবে।
6. Load test (হালকা): VM-এ SSH করে CPU load তৈরি করুন (`stress` ইনস্টল থাকলে, অথবা `for i in $(seq 1 4); do (while true; do :; done &) done` — সাবধানে, পরীক্ষা শেষে process থামান)। কিছুক্ষণ পর MIG-এ VM সংখ্যা বাড়ছে কি না দেখুন।

7. Rolling update পরীক্ষা: নতুন template version তৈরি করে MIG-এ **Rolling update** চালান (max surge 1, max unavailable 0)।

gcloud Commands

```
gcloud compute instance-templates create technova-web-template --machine-type=e2-micro --
network=technova-vpc --subnet=technova-web-subnet --tags=technova-web --image-family=debian-12
--image-project=debian-cloud --service-account=technova-app-sa@technova-
lab.iam.gserviceaccount.com --metadata-from-file=startup-script=CHANGE-ME-STARTUP-SCRIPT.sh
gcloud compute instance-groups managed create technova-web-mig --base-instance-name=technova-
web-mig --size=1 --template=technova-web-template --region=asia-south1 --health-
check=technova-web-health --initial-delay=300
gcloud compute instance-groups managed set-autoscaling technova-web-mig --region=asia-south1
--min-num-replicas=1 --max-num-replicas=3 --target-cpu-utilization=0.6 --cool-down-period=60
gcloud compute instance-groups managed list-instances technova-web-mig --region=asia-south1
gcloud compute instance-groups managed describe technova-web-mig --region=asia-south1
```

Expected Output

```
# Illustrative output
# NAME                     ZONE           STATUS   ACTION
# technova-web-mig-abc1    asia-south1-a  RUNNING  NONE
# CPU load বাড়লে list-instances-এ 2–3টি instance দেখা যাবে।
```

Verification

- MIG-এ কমপক্ষে 1টি RUNNING instance আছে কি না দেখুন।
- Autoscaler status: `gcloud compute instance-groups managed describe`-এ autoscaling enabled ও min/max যাচাই করুন।
- একটি instance মুছে ফেলুন (`delete-instances`) — MIG স্বয়ংক্রিয়ভাবে নতুন instance তৈরি করে কি না দেখুন (autohealing)।

Common Errors

Error	কারণ	সমাধান
MIG instances UNHEALTHY	Health check firewall বন্ধ	Lab 9-এর health check IP range rule যাচাই করুন
Autoscaling not triggering	Cooldown বা target খুব বেশি	Target 60% ও cooldown 60s যাচাই করুন; কয়েক মিনিট অপেক্ষা করুন
Quota exceeded on scale-out	Max replicas quota ছাড়িয়েছে	Max 3-এ সীমিত রাখুন; quota যাচাই করুন

Troubleshooting

- MIG-এর **Errors** ট্যাব দেখুন — instance তৈরি ব্যর্থ হলে কারণ সেখানে লেখা থাকে।
- Startup script-এর log serial console-এ দেখুন।

Security Considerations

- Template-এ service account-এর least-privilege role দিন।
- Max replicas সীমা দিন — ভুল autoscaling-এ খরচ যেন অসীম না বাড়ে।
- Instance-এ public IP না দিয়ে load balancer-এর পেছনে রাখুন (Lab 9 + Lab 12)।

Cleanup and Deletion

- MIG মুছুন: `gcloud compute instance-groups managed delete technova-web-mig --region=asia-south1` (এতে VM-গুলোও মুছে যাবে)।
- Instance template মুছুন: `gcloud compute instance-templates delete technova-web-template`।
- Health check রেখে দিন বা মুছুন — Lab 12-এ আবার লাগলে তৈরি করবেন।

Reflection Questions

1. Autoscaling-এর target CPU 60% বনাম 80% — কোনটি কখন ভালো?
2. Autohealing ও autoscaling-এর পার্থক্য কী?
3. Rolling update-এ max surge ও max unavailable-এর trade-off কী?

ল্যাব 11 • Lab Workbook

Lab 11 — Cloud Storage Bucket Security and Lifecycle

Learning Objectives

- Cloud Storage bucket তৈরি করে uniform bucket-level access চালু করতে পারবেন।
- Versioning, lifecycle rule ও retention policy সেট করতে পারবেন।
- Public access প্রতিরোধ ও signed URL-এর ধারণা ব্যাখ্যা করতে পারবেন।

Prerequisites

- Lab 2 (service account), Lab 3 (gcloud) সম্পন্ন।

Required Services

- Cloud Storage।

Estimated Duration

৪৫–৬০ মিনিট।

Cost-Risk Level

Low — ছোট test file-এর storage খরচ নগণ্য; lifecycle দিয়ে পুরোনো version মুছুন।

Console Instructions

1. Console → **Cloud Storage** → **Buckets** → **Create**। নাম: `technova-lab-assets-CHANGE-ME-UNIQUE` (globally unique হতে হবে)। Location: `asia-south1`। Storage class: Standard।
2. Access control: **Uniform** নির্বাচন করুন। Public access prevention: **Enforced**।
3. Protection: Object versioning **On** করুন। Retention policy: এই ল্যাবে সেট করবেন না (পরীক্ষার জন্য), তবে production-এ বিবেচনা করুন।
4. একটি test file আপলোড করুন: `hello.txt` (লেখা: "Hello from TechNova Storage")।
5. **Lifecycle:** Bucket → Lifecycle → Add rule। Rule: object-এর বয়স 30 দিন হলে Nearline-এ যাবে; 90 দিন হলে Delete হবে। Noncurrent version 7 দিন পর Delete হবে।
6. Service account-কে access দিন: Permissions → Grant access → `technova-app-sa`-কে `Storage Object Viewer`।
7. Public access পরীক্ষা: file-এর public URL ব্রাউজারে খুললে 403 আসা উচিত — এটাই প্রত্যাশিত।

gcloud Commands

```
gsutil mb -p technova-lab -c STANDARD -l asia-south1 -b on gs://technova-lab-assets-CHANGE-ME-UNIQUE/
gsutil uniformbucketlevelaccess set on gs://technova-lab-assets-CHANGE-ME-UNIQUE/
gsutil versioning set on gs://technova-lab-assets-CHANGE-ME-UNIQUE/
echo "Hello from TechNova Storage" > hello.txt
gsutil cp hello.txt gs://technova-lab-assets-CHANGE-ME-UNIQUE/
gsutil ls -l gs://technova-lab-assets-CHANGE-ME-UNIQUE/
gcloud storage buckets update gs://technova-lab-assets-CHANGE-ME-UNIQUE/ --public-access-prevention
cat > lifecycle.json <<'EOF'
{
  "rule": [
    {"action": {"type": "SetStorageClass", "storageClass": "NEARLINE"}, "condition": {"age": 30}},
    {"action": {"type": "Delete"}, "condition": {"age": 90}},
    {"action": {"type": "Delete"}, "condition": {"isLive": false, "age": 7}}
  ]
}
EOF
gsutil lifecycle set lifecycle.json gs://technova-lab-assets-CHANGE-ME-UNIQUE/
```

> **সতর্কতা:** Bucket-এর নাম globally unique। নিজের নাম বা প্রজেক্ট ID দিয়ে unique suffix দিন। Public access prevention চালু থাকলে ভুল করে public file তৈরি হলেও তা public হবে না।

Expected Output

```
# Illustrative output
# $ gsutil ls -l gs://technova-lab-assets-CHANGE-ME-UNIQUE/
# 32  2026-11-09T...  gs://technova-lab-assets-CHANGE-ME-UNIQUE/hello.txt
# Public URL-এ 403 Forbidden আসা প্রত্যাশিত।
```

Verification

- `gsutil ls`-এ file দেখা যাচ্ছে কি না দেখুন।
- `gsutil lifecycle get`-এ তিনটি rule আছে কি না যাচাই করুন।
- Versioning On কি না: `gsutil versioning get`।
- অন্য account থেকে (বা logged-out ব্রাউজারে) file খুললে access denied হয় কি না পরীক্ষা করুন।

Common Errors

Error	কারণ	সমাধান
Bucket name already exists	নাম globally unique নয়	Unique suffix যোগ করুন
AccessDenied on upload	Storage Object Creator role নেই	নিজের account-এ role যাচাই করুন

Lifecycle JSON invalid	JSON syntax ভুল	JSON validator দিয়ে যাচাই করুন

Troubleshooting

- `gsutil` না থাকলে `gcloud storage` command ব্যবহার করুন (নতুন CLI)।
- Bucket location পরিবর্তন করা যায় না — ভুল location হলে নতুন bucket তৈরি করতে হয়।

Security Considerations

- Uniform bucket-level access ব্যবহার করুন — object-level ACL এড়িয়ে চলুন।
- Public access prevention Enforced রাখুন।
- সংবেদনশীল file-এর জন্য signed URL ব্যবহার করুন, public link নয়।
- CMEK দিয়ে encryption বিবেচনা করুন সংবেদনশীল ডেটার জন্য।

Cleanup and Deletion

- Test file মুছুন: `gsutil rm gs://technova-lab-assets-CHANGE-ME-UNIQUE/hello.txt`।
- Bucket মুছুন (পরের ল্যাবে আবার লাগলে রেখে দিতে পারেন — খরচ নগণ্য, তবে নাম নোট রাখুন): `gsutil rm -r gs://technova-lab-assets-CHANGE-ME-UNIQUE/` — versioning On থাকলে সব version মুছতে `gsutil -m rm -r` ব্যবহার করুন।

Reflection Questions

1. Uniform বনাম Fine-grained access — কখন কোনটি?
2. Lifecycle rule কীভাবে খরচ কমায়? একটি উদাহরণ দিন।
3. Versioning চালু থাকলে ভুল করে file মুছে গেলে কীভাবে ফেরত পাবেন?

ল্যাব 12 • Lab Workbook

Lab 12 — Cloud DNS, HTTPS and Cloud CDN

Learning Objectives

- Cloud DNS managed zone ও DNS record তৈরি করতে পারবেন।
- Google-managed SSL certificate দিয়ে HTTPS load balancer সেট করতে পারবেন।
- Cloud CDN enable করে cache আচরণ যাচাই করতে পারবেন।

Prerequisites

- Lab 9 (load balancer) সম্পন্ন। একটি test ডোমেইন বা `technova.example` (কাল্পনিক) ব্যবহারের ধারণা জানা।

Required Services

- Cloud DNS, Cloud Load Balancing, Cloud CDN, Certificate Manager (Google-managed certificate)।

Estimated Duration

৯০ মিনিট (DNS propagation ও certificate provisioning-এ সময় লাগতে পারে)।

Cost-Risk Level

Medium — Load balancer, DNS zone ও CDN billable; ল্যাব শেষে মুছুন।

Console Instructions

1. Console → **Network services** → **Cloud DNS** → **Create Zone**। Zone name: `technova-example-zone`, DNS name: `technova.example`, DNSSEC: Off (ল্যাবের জন্য)।
2. Lab 9-এর load balancer আবার তৈরি করুন (বা বিদ্যমানটি ব্যবহার করুন) এবং তার external IP নোট করুন।
3. DNS-এ A record যোগ করুন: নাম `www.technova.example`, type A, TTL 300, value: load balancer-এর IP।
4. **HTTPS:** Load balancing → Load balancer → Edit। Frontend-এ নতুন HTTPS frontend যোগ করুন: protocol HTTPS, port 443, IP: একই বা নতুন। Certificate: **Create a new certificate** → Google-managed certificate, domains: `www.technova.example`।
5. Backend service-এ **Cloud CDN: Enable** করুন। Cache mode: Cache static content।
6. HTTP থেকে HTTPS-এ redirect: URL map-এ HTTP frontend-এর জন্য redirect rule যোগ করুন (অথবা HTTP frontend রেখে দিন — ল্যাবে redirect ঐচ্ছিক)।
7. Certificate status PROVISIONING থেকে ACTIVE হতে সময় লাগতে পারে — ধৈর্য ধরুন। DNS record সঠিকভাবে LB IP-তে point করছে কি না যাচাই করুন।

> **নোট — কাল্পনিক ডোমেইন:** `technova.example` একটি কাল্পনিক ডোমেইন — আসল DNS-এ এটি resolve হবে না। এই ল্যাবে DNS record ও certificate-এর কনফিগারেশন শেখাই লক্ষ্য। আসল ডোমেইন থাকলে সেই ডোমেইন দিয়ে একই ধাপ করুন; না থাকলে `nip.io` ধরনের test ডোমেইন বা শুধু LB IP দিয়ে HTTPS ধাপটি বুঝে নিন। Certificate ACTIVE হতে DNS validation লাগে — কাল্পনিক ডোমেইনে তা সম্পূর্ণ হবে না, সেটি প্রত্যাশিত।

gcloud Commands

```
gcloud dns managed-zones create technova-example-zone --dns-name=technova.example --
description="TechNova lab DNS zone"
gcloud dns record-sets create www.technova.example --zone=technova-example-zone --type=A --
ttl=300 --rrdatas=203.0.113.30
gcloud dns record-sets list --zone=technova-example-zone
gcloud compute ssl-certificates create technova-www-cert --domains=www.technova.example --
global
gcloud compute backend-services update technova-web-backend --enable-cdn --global
gcloud compute target-https-proxies create technova-web-https-proxy --url-map=technova-web-map
--ssl-certificates=technova-www-cert
gcloud compute forwarding-rules create technova-lb-https-frontend --global --target-https-
proxy=technova-web-https-proxy --ports=443
```

Expected Output

```
# Illustrative output
# $ gcloud dns record-sets list --zone=technova-example-zone
# NAME                  TYPE  TTL  DATA
# www.technova.example. A     300  203.0.113.30
# $ curl -I http://CHANGE-ME-LB-IP
# HTTP/1.1 200 OK
# CDN header-এ cache status (HIT/MISS) দেখা যাবে।
```

Verification

- `dig www.technova.example` বা `nslookup`-এ A record LB IP দেখাচ্ছে কি না দেখুন (আসল ডোমেইন থাকলে)।
- Backend service-এ CDN enabled কি না: `gcloud compute backend-services describe technova-web-backend --global`।
- দুইবার `curl` করে response header-এ `Age` / cache status পরিবর্তন হচ্ছে কি না দেখুন।
- Certificate status: `gcloud compute ssl-certificates describe technova-www-cert --global`।

Common Errors

Error	কারণ	সমাধান
Certificate stuck in PROVISIONING	DNS validation হয়নি	DNS record LB IP-তে point করছে কি না যাচাই করুন; সময় দিন
DNS not resolving	Nameserver পরিবর্তন হয়নি	

		Registrar-এ Cloud DNS nameserver বসাতে হয় (আসল ডোমেইনে)
CDN not caching	Cache-Control header বা dynamic content	Static file দিয়ে পরীক্ষা করুন; cache mode যাচাই করুন

Troubleshooting

- DNS propagation-এ ৫–৩০ মিনিট লাগতে পারে — `dig @8.8.8.8` দিয়ে Google DNS থেকে যাচাই করুন।
- HTTPS পরীক্ষার জন্য `curl -k` ব্যবহার করবেন না production-এ; ল্যাবে certificate mismatch বুঝতে `curl -v` দেখুন।

Security Considerations

- Production-এ সব traffic HTTPS-এ redirect করুন; HSTS header বিবেচনা করুন।
- Google-managed certificate auto-renew হয় — manual certificate-এর expiry মনে রাখতে হয়।
- DNSSEC চালু করলে DNS spoofing-এর ঝুঁকি কমে।

Cleanup and Deletion

- HTTPS frontend, proxy, certificate মুছুন।
- DNS record ও managed zone মুছুন: `gcloud dns managed-zones delete technova-example-zone` (আগে record মুছুন)।
- Load balancer, backend ও VM মুছুন (Lab 9-এর cleanup ধাপ অনুসরণ করুন)।
- CDN disable করে backend service মুছুন।

Reflection Questions

1. Google-managed certificate বনাম self-managed certificate — কখন কোনটি?
2. CDN কখন খরচ কমায়, কখন বাড়ায়?
3. DNS TTL ছোট রাখার সুবিধা ও অসুবিধা কী?

ল্যাব 13 • Lab Workbook

Lab 13 — Cloud SQL Database Deployment

Learning Objectives

- Cloud SQL (PostgreSQL) instance তৈরি করতে পারবেন — private IP সহ।
- Database, user ও password (Secret Manager-এ) তৈরি করতে পারবেন।
- VM বা Cloud Run থেকে private সংযোগ পরীক্ষা করতে পারবেন।
- Automated backup ও point-in-time recovery চালু করতে পারবেন।

Prerequisites

- Lab 2 (service account), Lab 4 (VPC), Lab 6 (PSC ধারণা) সম্পন্ন।

Required Services

- Cloud SQL, Secret Manager, Service Networking API, Compute Engine (test VM)।

Estimated Duration

৯০ মিনিট (instance তৈরি হতে ১০–২০ মিনিট লাগতে পারে)।

Cost-Risk Level

Medium–High — Cloud SQL instance চলমান থাকলেই চার্জ হয়; সবচেয়ে ছোট tier ব্যবহার করুন ও ল্যাব শেষে মুছুন।

Console Instructions

1. **Private Service Access সেট করুন:** Console → VPC network → Private Service Access → Allocate IP range (নাম `technova-sql-range`, range `10.20.0.0/16`) → Create connection। এটি একবারই করতে হয়।
2. Console → **SQL** → **Create Instance** → **PostgreSQL**। Instance ID: `technova-db`, password: একটি শক্তিশালী password তৈরি করুন (Console-এর generate ব্যবহার করুন)।
3. Version: PostgreSQL 15 (বা উপলব্ধ সর্বশেষ)। Region: `asia-south1`, Zone: `asia-south1-a`। Machine: সবচেয়ে ছোট (Shared core, 1 vCPU)। Storage: 10 GB SSD, automatic storage increase On।
4. Connectivity: **Private IP** নির্বাচন করুন, network `technova-vpc`, allocated range `technova-sql-range`। Public IP **Off** রাখুন।
5. Backups: Automated backups **On**, Point-in-time recovery **On**, retention 7 দিন।
6. **Create Instance** ক্লিক করুন। তৈরি হতে সময় লাগবে।
7. Instance তৈরি হলে **Databases** → **Create Database**: নাম `technova_app`। **Users** → **Add User Account**: username `technova_app_user`, password আলাদা শক্তিশালী password।

8. Password Secret Manager-এ সংরক্ষণ করুন: Secret Manager → Create Secret, নাম `technova-db-password`, value-তে password দিন। Code-এ বা terminal-এ password লিখে রাখবেন না।
9. সংযোগ পরীক্ষা: Lab 7-এর VM (বা নতুন test VM) থেকে `psql` দিয়ে সংযোগ করুন — VM-এ `postgresql-client` ইনস্টল করুন।

gcloud Commands

```
gcloud services enable servicenetworking.googleapis.com sqladmin.googleapis.com
secretmanager.googleapis.com
gcloud compute addresses create technova-sql-range --global --purpose=VPC_PEERING --prefix-
length=16 --network=technova-vpc
gcloud services vpc-peerings connect --service=servicenetworking.googleapis.com --
ranges=technova-sql-range --network=technova-vpc --project=technova-lab
gcloud sql instances create technova-db --database-version=POSTGRES_15 --region=asia-south1 --
tier=db-f1-micro --storage-size=10GB --storage-type=SSD --network=technova-vpc --no-assign-ip
--root-password=CHANGE-ME-STRONG-PASSWORD --backup-start-time=02:00 --enable-point-in-time-
recovery --retained-backups-count=7
gcloud sql databases create technova_app --instance=technova-db
gcloud sql users create technova_app_user --instance=technova-db --password=CHANGE-ME-STRONG-
PASSWORD-2
gcloud sql instances describe technova-db --format="get(ipAddresses[0].ipAddress)"
echo -n "CHANGE-ME-STRONG-PASSWORD-2" | gcloud secrets create technova-db-password --data-
file=-
```

> **সতর্কতা — Password:** উপরের command-এ `CHANGE-ME-STRONG-PASSWORD` একটি placeholder। আসল password terminal history-তে থেকে যায় — তাই command চালানোর পর history মুছুন অথবা Console থেকে password সেট করুন। Production-এ password কখনো command line-এ লিখবেন না।

Expected Output

```
# Illustrative output
# $ gcloud sql instances describe technova-db
# ipAddresses:
# - ipAddress: 10.20.0.5
#   type: PRIVATE
# VM থেকে psql-এ সংযোগ হলে:
# technova_app=> SELECT version();
# PostgreSQL 15.x ...
```

Verification

- Instance state RUNNABLE কি না দেখুন।
- Private IP বরাদ্দ হয়েছে কি না যাচাই করুন; public IP নেই তা নিশ্চিত করুন।
- VM থেকে `psql -h 10.20.0.5 -U technova_app_user -d technova_app` দিয়ে সংযোগ ও একটি test table তৈরি করে দেখুন।
- Backup configuration On কি না যাচাই করুন।

Common Errors

Error	কারণ	সমাধান
Connection timed out	VM ও SQL একই VPC-তে নেই বা private service access হয়নি	VPC peering ও allocated range যাচাই করুন
Password authentication failed	ভুল user/password	Secret Manager থেকে সঠিক password নিন
Instance creation failed	Tier/region quota	সবচেয়ে ছোট tier ও `asia-south1` ব্যবহার করুন

Troubleshooting

- `gcloud sql operations list --instance=technova-db` দিয়ে instance তৈরির operation-এর status দেখুন।
- Cloud SQL Auth Proxy ব্যবহার করলে public IP ছাড়াও নিরাপদ সংযোগ করা যায় — Lab 15-এ এটি আবার আসবে।

Security Considerations

- Public IP বন্ধ রাখুন; শুধু private IP ব্যবহার করুন।
- Password Secret Manager-এ রাখুন; code-এ hardcode করবেন না।
- Service account-কে শুধু `Cloud SQL Client` role দিন।
- SSL/TLS সংযোগ চালু করুন production-এ।

Cleanup and Deletion

- Instance মুছুন: `gcloud sql instances delete technova-db` (deletion protection On থাকলে আগে Off করুন)।
- Secret মুছুন বা রেখে দিন (পরের ল্যাবে লাগলে): `gcloud secrets delete technova-db-password`।
- Private service access connection রেখে দিতে পারেন — পরের ল্যাবে আবার লাগবে; না লাগলে মুছুন।

Reflection Questions

1. Cloud SQL-এ private IP ব্যবহারের সুবিধা কী? Public IP কখন দরকার হয়?
2. Point-in-time recovery কীভাবে ডেটা হারানো কমায়?
3. Database password Secret Manager-এ রাখা কেন code-এ রাখার চেয়ে নিরাপদ?

ল্যাব 14 • Lab Workbook

Lab 14 — Firestore Application Integration

Learning Objectives

- Firestore database (Native mode) তৈরি করতে পারবেন।
- Node.js অ্যাপ থেকে Firestore-এ document লেখা ও পড়া করতে পারবেন।
- Security Rules দিয়ে read/write নিয়ন্ত্রণ করতে পারবেন।
- Collection ও document নকশা ব্যাখ্যা করতে পারবেন।

Prerequisites

- Lab 3 (gcloud/ADC), Node.js ইনস্টল করা। Lab 13-এর database ধারণা জানা।

Required Services

- Firestore, Cloud Run (ঐচ্ছিক, test-এর জন্য local)।

Estimated Duration

৬০ মিনিট।

Cost-Risk Level

Low — ছোট test ডেটায় Firestore-এর free tier-এর মধ্যে থাকার সম্ভাবনা বেশি; তবুও usage যাচাই করুন।

Console Instructions

1. Console → **Firestore** → **Create Database**। Mode: **Native mode**। Location: `asia-south1`।
2. Security Rules: শুরুতে **Test mode** নয় — **Production mode** (deny all) নির্বাচন করুন, তারপর নিচের rule লিখুন (শুধু authenticated read, কোনো write নয় — ল্যাবের জন্য):

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /tasks/{taskId} {
      allow read: if true;
      allow write: if false;
    }
  }
}
```

1. Console → **Firestore** → **Data** → **Start Collection**। Collection ID: `tasks`। একটি document যোগ করুন: field `title` = "Deploy TechNova API", `done` = false, `createdAt` = timestamp।

2. নিজের কম্পিউটারে একটি ছোট Node.js script লিখুন (নিচে দেওয়া হয়েছে) যা Firestore থেকে tasks পড়ে।
3. `GOOGLE_APPLICATION_CREDENTIALS` সেট না করে `gcloud auth application-default login` দিয়ে ADC ব্যবহার করুন (Lab 3)।

gcloud Commands

```
gcloud firestore databases create --location=asia-south1 --type=firestore-native
gcloud firestore collections list
npm init -y
npm install @google-cloud/firestore
node read-tasks.js
```

```
// read-tasks.js — Illustrative example
const { Firestore } = require('@google-cloud/firestore');
const db = new Firestore({ projectId: 'technova-lab' });
async function main() {
  const snap = await db.collection('tasks').get();
  snap.forEach((doc) => console.log(doc.id, doc.data()));
}
main().catch(console.error);
```

Expected Output

```
# Illustrative output
# $ node read-tasks.js
# CHANGE-ME-DOC-ID { title: 'Deploy TechNova API', done: false, createdAt: ... }
```

Verification

- Console-এ `tasks` collection ও document দেখা যাচ্ছে কি না দেখুন।
- Script চালিয়ে document পড়া যাচ্ছে কি না যাচাই করুন।
- Write চেষ্টা করলে Security Rules deny করছে কি না পরীক্ষা করুন (প্রত্যাশিত: deny)।
- `gcloud firestore operations list`-এ database ACTIVE কি না দেখুন।

Common Errors

Error	কারণ	সমাধান
PERMISSION_DENIED on read	Security Rules deny করছে	Rule যাচাই করুন; ADC account-এ Firestore Viewer role দিন
Database already exists	আগে তৈরি করা হয়েছে	বিদ্যমান database ব্যবহার করুন
Could not load default credentials	ADC সেট হয়নি	`gcloud auth application-default login` চালান

Troubleshooting

- Rules Playground (Console → Firestore → Rules → Playground) দিয়ে rule পরীক্ষা করুন।
- Location একবার সেট হলে পরিবর্তন করা যায় না — ভুল হলে database মুছে আবার তৈরি করতে হয়।

Security Considerations

- Test mode-এ (allow read/write if true) কখনো production database রাখবেন না।
- Rule-এ শুধু প্রয়োজনীয় collection ও operation allow করুন।
- সংবেদনশীল ডেটা Firestore-এ রাখলে encryption ও access control যাচাই করুন।

Cleanup and Deletion

- Test document মুছুন: Console থেকে `tasks` collection-এর document delete করুন, অথবা script দিয়ে delete করুন।
- Firestore database রেখে দিতে পারেন (খরচ নগণ্য) — Lab 24-এ আবার ব্যবহার হতে পারে। সম্পূর্ণ মুছতে: `gcloud firestore databases delete --database="(default)"` (সব ডেটা মুছে যাবে — নিশ্চিত হয়ে করুন)।

Reflection Questions

1. Firestore (NoSQL) বনাম Cloud SQL (relational) — TechNova-র কোন ডেটার জন্য কোনটি ভালো?
2. Security Rules-এ `allow write: if false` রাখার মানে কী?
3. Collection নকশায় nested subcollection কখন ব্যবহার করবেন?

ল্যাব 15 • Lab Workbook

Lab 15 — Cloud Run Node.js Deployment

Learning Objectives

- Node.js + Express অ্যাপ তৈরি করে container image বানাতে পারবেন।
- Artifact Registry-তে image push করে Cloud Run-এ deploy করতে পারবেন।
- Environment variable ও Secret Manager দিয়ে configuration দিতে পারবেন।
- Cloud Run-এর scaling ও revision ধারণা ব্যাখ্যা করতে পারবেন।

Prerequisites

- Lab 3 (gcloud), Lab 13 (Cloud SQL ধারণা), Lab 17-এর Artifact Registry ধারণা (এই ল্যাবে সংক্ষেপে করা হবে)। Node.js ও Docker জানা।

Required Services

- Cloud Run, Artifact Registry, Secret Manager, Cloud Build (ঐচ্ছিক)।

Estimated Duration

৯০ মিনিট।

Cost-Risk Level

Low–Medium — Cloud Run scale-to-zero থাকলে খরচ কম; তবুও min-instances 0 রাখুন।

Console Instructions

1. নিজের কম্পিউটারে একটি ফোল্ডার তৈরি করুন `technova-api`। নিচের file-গুলো লিখুন:

```
// package.json
{
  "name": "technova-api",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": { "start": "node index.js" },
  "dependencies": { "express": "^4.19.0" }
}
// index.js
const express = require('express');
const app = express();
const port = process.env.PORT || 8080;
app.get('/', (req, res) => res.json({ service: 'technova-api', status: 'ok' }));
app.get('/health', (req, res) => res.json({ status: 'healthy' }));
app.listen(port, () => console.log(`Listening on ${port}`));
// Dockerfile
```

```
FROM node:20-slim
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY . .
CMD ["npm", "start"]
```

1. Artifact Registry-তে repository তৈরি করুন: Console → Artifact Registry → Create Repository, নাম `technova-images`, format Docker, location `asia-south1`।
2. Terminal-এ image build ও push করুন (নিচের command দেখুন)।
3. Console → **Cloud Run** → **Create Service**। নাম `technova-api`, region `asia-south1`, container image: Artifact Registry-র image URL। Authentication: **Allow unauthenticated invocations** (ল্যাবের public API-র জন্য; private API হলে Require authentication)।
4. Service account: `technova-app-sa`। Environment variable: `NODE_ENV=production`। Secret: `technova-db-password` থেকে `DB_PASSWORD` হিসেবে mount করুন (Lab 13-এর secret থাকলে)।
5. CPU allocation: **CPU is only allocated during request processing**। Min instances: 0, Max instances: 2। **Create** করুন।
6. Service URL নোট করুন। ব্রাউজারে খুলে `{"service":"technova-api","status":"ok"}` দেখুন।

gcloud Commands

```
gcloud artifacts repositories create technova-images --repository-format=docker --
location=asia-south1
gcloud auth configure-docker asia-south1-docker.pkg.dev
docker build -t asia-south1-docker.pkg.dev/technova-lab/technova-images/technova-api:v1 .
docker push asia-south1-docker.pkg.dev/technova-lab/technova-images/technova-api:v1
gcloud run deploy technova-api --image=asia-south1-docker.pkg.dev/technova-lab/technova-
images/technova-api:v1 --region=asia-south1 --allow-unauthenticated --service-
account=technova-app-sa@technova-lab.iam.gserviceaccount.com --set-env-
vars=NODE_ENV=production --max-instances=2 --min-instances=0
gcloud run services describe technova-api --region=asia-south1 --format="get(status.url)"
curl $(gcloud run services describe technova-api --region=asia-south1 --
format="get(status.url)")/health
```

Expected Output

```
# Illustrative output
# $ curl https://technova-api-CHANGE-ME-HASH.a.run.app/health
# {"status":"healthy"}
# $ curl https://technova-api-CHANGE-ME-HASH.a.run.app/
# {"service":"technova-api","status":"ok"}
```

Verification

- Service URL-এ `/` ও `/health` দুটোই 200 দিচ্ছে কি না দেখুন।
- Console → Cloud Run → Revisions-এ revision তৈরি হয়েছে কি না দেখুন।

- Log-এ "Listening on 8080" দেখা যাচ্ছে কি না: `gcloud run services logs read technova-api --region=asia-south1`।
- নতুন image push করে আবার deploy করলে নতুন revision তৈরি হয় কি না পরীক্ষা করুন।

Common Errors

Error	কারণ	সমাধান
Container failed to start	PORT env শোনা হয়নি বা app crash	`process.env.PORT` ব্যবহার করুন; log দেখুন
Image not found	Repository/region ভুল	Image URL যাচাই করুন
Permission denied on deploy	Cloud Run Admin role নেই	Role যাচাই করুন; service account-এর permission দেখুন

Troubleshooting

- Cloud Run log-এ error দেখুন — বেশিরভাগ সমস্যা সেখানে লেখা থাকে।
- Local-এ `docker run -p 8080:8080 IMAGE` দিয়ে image পরীক্ষা করুন deploy-এর আগে।

Security Considerations

- Secret কখনো environment variable-এ plain text-এ লিখবেন না — Secret Manager ব্যবহার করুন।
- Private API হলে unauthenticated access বন্ধ রাখুন।
- Max instances সীমা দিন — ভুল traffic-এ খরচ যেন অসীম না বাড়ে।

Cleanup and Deletion

- Service মুছুন: `gcloud run services delete technova-api --region=asia-south1`।
- Image মুছুন বা repository রেখে দিন (Lab 17, 21-এ লাগবে)। Image মুছতে: Artifact Registry থেকে `technova-api:v1` delete করুন।
- Secret রেখে দিন — Lab 24-এ লাগবে।

Reflection Questions

1. Cloud Run-এ min-instances 0 রাখার সুবিধা ও অসুবিধা (cold start) কী?
2. Revision কী? Rollback কীভাবে করবেন?
3. Container image-এ secret রাখা কেন বিপজ্জনক?

ল্যাব 16 • Lab Workbook

Lab 16 — Pub/Sub Event-Driven Integration

Learning Objectives

- Pub/Sub topic ও subscription তৈরি করতে পারবেন।
- Message publish ও pull করতে পারবেন।
- Cloud Run service-কে Pub/Sub push subscription দিয়ে trigger করতে পারবেন।
- Dead-letter topic ও retry নীতি ব্যাখ্যা করতে পারবেন।

Prerequisites

- Lab 15 সম্পন্ন (Cloud Run service)।

Required Services

- Pub/Sub, Cloud Run।

Estimated Duration

৬০ মিনিট।

Cost-Risk Level

Low — ছোট test message-এর খরচ নগণ্য।

Console Instructions

1. Console → **Pub/Sub** → **Topics** → **Create Topic**। Topic ID: `technova-orders`। Default subscription যোগ করুন: নাম `technova-orders-sub`, delivery type: Pull, acknowledgement deadline 10s, message retention 7 দিন।
2. একটি dead-letter topic তৈরি করুন: `technova-orders-dlq`। Subscription-এর **Dead lettering**-এ DLQ topic সেট করুন, max delivery attempts 5।
3. Terminal থেকে message publish করুন (নিচের command)।
4. Pull করে message দেখুন, তারপর acknowledge করুন।
5. **Push subscription:** Lab 15-এর Cloud Run service-এ একটি `/events` endpoint যোগ করুন (message log করে 200 দেয়), নতুন image deploy করুন, তারপর নতুন subscription তৈরি করুন: delivery type Push, endpoint: Cloud Run service URL + `/events`।
6. আবার message publish করুন — Cloud Run log-এ message পৌঁছেছে কি না দেখুন।

gcloud Commands

```
gcloud pubsub topics create technova-orders
gcloud pubsub topics create technova-orders-dlq
gcloud pubsub subscriptions create technova-orders-sub --topic=technova-orders --ack-
deadline=10 --message-retention-duration=7d --dead-letter-topic=technova-orders-dlq --max-
delivery-attempts=5
gcloud pubsub topics publish technova-orders --message='{"orderId":"ORD-1001","amount":
2500,"currency":"BDT"}'
gcloud pubsub subscriptions pull technova-orders-sub --auto-ack --limit=5
gcloud pubsub subscriptions create technova-orders-push --topic=technova-orders --push-
endpoint=https://CHANGE-ME-CLOUD-RUN-URL/events --ack-deadline=10
```

Push subscription-এর অনুমতি: Pub/Sub-কে Cloud Run service invoke করার অনুমতি দিতে হয়। Cloud Run service-এ `roles/run.invoker` দিন `serviceAccount:service-CHANGE-ME-PROJECT-NUMBER@gcp-sa-pubsub.iam.gserviceaccount.com`-কে (Pub/Sub service agent)। অনুমতি না দিলে push 403 দেবে।

Expected Output

```
# Illustrative output
# $ gcloud pubsub subscriptions pull technova-orders-sub --auto-ack --limit=5
# ┌──────────────────────────────────────┬──────────────────────┐
# | DATA                                 | MESSAGE_ID           |
# | {"orderId":"ORD-1001",...}           | CHANGE-ME-MESSAGE-ID |
# └──────────────────────────────────────┴──────────────────────┘
# Cloud Run log-এ /events-এ message পৌঁছানোর entry দেখা যাবে।
```

Verification

- Topic ও subscription list-এ দেখা যাচ্ছে কি না: `gcloud pubsub topics list`, `gcloud pubsub subscriptions list`।
- Pull-এ message পাওয়া যাচ্ছে কি না দেখুন।
- Push subscription-এ Cloud Run log-এ event আসছে কি না যাচাই করুন।
- DLQ configuration আছে কি না: `gcloud pubsub subscriptions describe technova-orders-sub`।

Common Errors

Error	কারণ	সমাধান
Push returns 403	Pub/Sub service agent-কে invoker role দেওয়া হয়নি	উপরের callout অনুযায়ী role দিন
Message not received	Subscription ভুল topic-এ	Topic ও subscription-এর সংযোগ যাচাই করুন
Duplicate delivery	Ack হয়নি বা deadline পার হয়েছে	

		Ack deadline বাড়ান; idempotent handler লিখুন

Troubleshooting

- Message delivery-এর metric Cloud Monitoring-এ দেখুন (topic-এর `subscription/num_undelivered_messages`)।
- Push endpoint-এ `curl` দিয়ে সরাসরি POST করে endpoint কাজ করে কি না পরীক্ষা করুন।

Security Considerations

- Push endpoint-এ authentication চালু রাখুন; unauthenticated push endpoint public-এ খোলা থাকলে যে কেউ message পাঠাতে পারে।
- Topic-এ publish করার permission শুধু প্রয়োজনীয় service account-কে দিন।
- Message-এ secret, password বা PII রাখবেন না।

Cleanup and Deletion

- Subscription মুছুন: `gcloud pubsub subscriptions delete technova-orders-sub technova-orders-push`।
- Topic মুছুন: `gcloud pubsub topics delete technova-orders technova-orders-dlq`।
- Cloud Run service Lab 15-এর cleanup অনুযায়ী মুছুন (বা Lab 24-এর জন্য রেখে দিন — নোট রাখুন)।

Reflection Questions

1. Pull বনাম Push subscription — কখন কোনটি ভালো?
2. Dead-letter topic কেন দরকার? Max delivery attempts 5 রাখার মানে কী?
3. Pub/Sub message at-least-once delivery দেয় — আপনার handler কীভাবে duplicate সামলাবে?

ল্যাব 17 • Lab Workbook

Lab 17 — Artifact Registry and Image Publishing

Learning Objectives

- Artifact Registry-তে Docker repository তৈরি ও পরিচালনা করতে পারবেন।
- Image tag, version ও vulnerability scanning ব্যাখ্যা করতে পারবেন।
- Image-এর access control (IAM) সেট করতে পারবেন।
- Cleanup policy দিয়ে পুরোনো image মুছতে পারবেন।

Prerequisites

- Lab 15 সম্পন্ন (Docker image build ও push)।

Required Services

- Artifact Registry, Container Scanning API (ঐচ্ছিক)।

Estimated Duration

৪৫ মিনিট।

Cost-Risk Level

Low — ছোট image-এর storage খরচ নগণ্য।

Console Instructions

1. Console → **Artifact Registry** → **Repositories**। Lab 15-এর `technova-images` repository দেখুন।
2. Repository-তে `technova-api` image-এর tag (`v1`) ও digest দেখুন।
3. নতুন version push করুন: image-এ ছোট পরিবর্তন করুন (যেমন `/health` response-এ version যোগ করুন), `v2` tag দিয়ে build ও push করুন।
4. **Vulnerability scanning:** Repository settings-এ scanning চালু আছে কি না দেখুন; image-এর vulnerability report দেখুন (যদি উপলব্ধ থাকে)।
5. **Cleanup policy:** Repository → Settings → Cleanup policies → Create policy। নিয়ম: `v1` -এর মতো পুরোনো tag 30 দিন পর মুছে যাবে, অথবা সর্বশেষ 5টি version রেখে বাকি মুছে যাবে।
6. **Access:** Repository-এর Permissions-এ `technova-app-sa` -কে `Artifact Registry Reader` দিন; অন্য কাউকে Writer দেবেন না।

gcloud Commands

```
gcloud artifacts repositories list --location=asia-south1
gcloud artifacts docker images list asia-south1-docker.pkg.dev/technova-lab/technova-images/
technova-api
docker build -t asia-south1-docker.pkg.dev/technova-lab/technova-images/technova-api:v2 .
docker push asia-south1-docker.pkg.dev/technova-lab/technova-images/technova-api:v2
gcloud artifacts docker images describe asia-south1-docker.pkg.dev/technova-lab/technova-
images/technova-api:v2
gcloud artifacts repositories add-iam-policy-binding technova-images --location=asia-south1 --
member="serviceAccount:technova-app-sa@technova-lab.iam.gserviceaccount.com" --role="roles/
artifactregistry.reader"
```

Expected Output

```
# Illustrative output
# $ gcloud artifacts docker images list .../technova-api
# IMAGE  DIGEST        TAGS
# ...    sha256:abc... v1
# ...    sha256:def... v2
```

Verification

- দুটি tag (`v1`, `v2`) list-এ দেখা যাচ্ছে কি না দেখুন।
- Digest দিয়ে image pull করা যায় কি না পরীক্ষা করুন: `docker pull ...@sha256:CHANGE-ME-DIGEST`।
- Cleanup policy তৈরি হয়েছে কি না যাচাই করুন।
- Reader role শুধু `technova-app-sa`-কে দেওয়া হয়েছে কি না দেখুন।

Common Errors

Error	কারণ	সমাধান
denied: permission denied on push	Artifact Registry Writer role নেই	`gcloud auth configure-docker` ও role যাচাই করুন
Repository not found	Location/নাম ভুল	`asia-south1` ও repository নাম যাচাই করুন
Image pull fails in Cloud Run	Service account-কে Reader role দেওয়া হয়নি	Reader binding যোগ করুন

Troubleshooting

- `docker login` সমস্যায় `gcloud auth configure-docker asia-south1-docker.pkg.dev` আবার চালান।
- Tag-এর বদলে digest ব্যবহার করুন production deploy-এ — tag পরিবর্তন হতে পারে, digest অপরিবর্তনীয়।

Security Considerations

- Image-এ secret রাখবেন না; vulnerability scan-এর ফলাফল দেখে base image update করুন।
- Production deploy-এ digest ব্যবহার করুন, `latest` tag নয়।
- Repository-এ public access দেবেন না।

Cleanup and Deletion

- Test image `v1` মুছুন (cleanup policy থাকলে তা স্বয়ংক্রিয়ভাবে হবে): `gcloud artifacts docker images delete .../technova-api:v1 --delete-tags`।
- Repository রেখে দিন — Lab 21-এর CI/CD-তে লাগবে। সম্পূর্ণ মুছতে: `gcloud artifacts repositories delete technova-images --location=asia-south1`।

Reflection Questions

1. Tag বনাম digest — production deploy-এ কোনটি নিরাপদ, কেন?
2. Cleanup policy না থাকলে Artifact Registry-র খরচ কীভাবে বাড়ে?
3. Container Registry (পুরোনো) বনাম Artifact Registry — কোনটি ব্যবহার করবেন?

ল্যাব 18 • Lab Workbook

Lab 18 — GKE and Kubernetes Deployment

Learning Objectives

- GKE Autopilot বা Standard cluster তৈরি করতে পারবেন।
- Deployment, Service ও Ingress দিয়ে অ্যাপ চালাতে পারবেন।
- `kubectl` দিয়ে workload যাচাই ও log দেখতে পারবেন।
- Workload Identity ধারণা ব্যাখ্যা করতে পারবেন।

Prerequisites

- Lab 4 (VPC), Lab 15 (container image), Lab 17 (Artifact Registry) সম্পন্ন। `kubectl` ধারণা জানা।

Required Services

- Google Kubernetes Engine (GKE), Artifact Registry, Compute Engine।

Estimated Duration

৯০–১২০ মিনিট (cluster তৈরি হতে ১০–২০ মিনিট লাগতে পারে)।

Cost-Risk Level

High — GKE cluster ও node billable; Autopilot-এ শুধু চলমান pod-এর জন্য চার্জ হয়। ল্যাব শেষে cluster মুছে ফেলুন।

Console Instructions

1. Console → **Kubernetes Engine** → **Clusters** → **Create**। Mode: **Autopilot** (ল্যাবের জন্য সহজ) — নাম `technova-gke`, region `asia-south1`, network `technova-vpc`, subnet `technova-web-subnet`।
2. Cluster তৈরি হলে **Connect** ক্লিক করে `kubectl` command copy করুন, terminal-এ চালান।
3. নিচের manifest file (`technova-app.yaml`) তৈরি করুন:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: technova-api
  namespace: default
spec:
  replicas: 2
  selector:
    matchLabels: { app: technova-api }
  template:
    metadata:
```

```
      labels: { app: technova-api }
    spec:
      containers:
        - name: technova-api
          image: asia-south1-docker.pkg.dev/technova-lab/technova-images/technova-api:v2
          ports: [{ containerPort: 8080 }]
          resources:
            requests: { cpu: "250m", memory: "256Mi" }
            limits: { cpu: "500m", memory: "512Mi" }
          livenessProbe:
            httpGet: { path: /health, port: 8080 }
            initialDelaySeconds: 10
            periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: technova-api
spec:
  type: ClusterIP
  selector: { app: technova-api }
  ports: [{ port: 80, targetPort: 8080 }]
```

1. Apply করুন: `kubectl apply -f technova-app.yaml`।
2. Pod ও Service যাচাই করুন: `kubectl get pods,svc`।
3. Port-forward করে পরীক্ষা করুন: `kubectl port-forward svc/technova-api 8080:80`, তারপর `curl http://localhost:8080/health`।
4. Scaling পরীক্ষা: `kubectl scale deployment technova-api --replicas=3`।
5. Workload Identity: Cluster-এর Workload Identity চালু আছে কি না দেখুন (Autopilot-এ default চালু থাকে)। Service account-কে Kubernetes service account-এর সঙ্গে যুক্ত করার ধারণা নোট করুন — এই ল্যাবে শুধু ধারণা, পূর্ণ সেটআপ Project 03-এ।

gcloud Commands

```
gcloud container clusters create-auto technova-gke --region=asia-south1 --network=technova-vpc
--subnetwork=technova-web-subnet
gcloud container clusters get-credentials technova-gke --region=asia-south1
kubectl get nodes
kubectl apply -f technova-app.yaml
kubectl get pods,svc,deployments
kubectl logs -l app=technova-api --tail=20
kubectl scale deployment technova-api --replicas=3
kubectl get pods -w
```

Expected Output

```
# Illustrative output
# NAME                           READY   STATUS    RESTARTS
# technova-api-abc12-xxxxx       1/1     Running   0
# technova-api-abc12-yyyyy       1/1     Running   0
```

```
# $ curl http://localhost:8080/health
# {"status":"healthy"}
```

Verification

- সব pod Running ও Ready 1/1 কি না দেখুন।
- Port-forward-এ `/health` 200 দিচ্ছে কি না যাচাই করুন।
- Scale করে 3টি pod হচ্ছে কি না দেখুন।
- একটি pod মুছে ফেলুন (`kubectl delete pod`) — Deployment নতুন pod তৈরি করে কি না দেখুন।

Common Errors

Error	কারণ	সমাধান
ImagePullBackOff	Image URL ভুল বা permission নেই	Artifact Registry Reader role ও image URL যাচাই করুন
CrashLoopBackOff	App PORT শুনছে না বা crash করছে	`kubectl logs` দেখুন; PORT env যাচাই করুন
Pod Pending	Resource quota বা node capacity	Requests কমান; cluster status দেখুন

Troubleshooting

- `kubectl describe pod CHANGE-ME-POD-NAME` দিয়ে event দেখুন — বেশিরভাগ সমস্যার কারণ সেখানে লেখা থাকে।
- GKE-র **Workloads** পেজে pod-এর status ও log দেখুন।

Security Considerations

- Private cluster ব্যবহার করুন production-এ — control plane ও node-এর public endpoint বন্ধ রাখুন।
- Workload Identity ব্যবহার করুন — node service account-এর broad permission এড়িয়ে চলুন।
- Resource limit দিন — একটি pod যেন সব resource খেয়ে না ফেলে।
- Image-এ vulnerability scan চালু রাখুন।

Cleanup and Deletion

- Workload মুছুন: `kubectl delete -f technova-app.yaml` ।
- Cluster মুছুন: `gcloud container clusters delete technova-gke --region=asia-south1` — এটি সবচেয়ে গুরুত্বপূর্ণ cleanup; cluster রেখে গেলে খরচ চলতে থাকবে।
- Cluster মুছে গেছে কি না `gcloud container clusters list` দিয়ে যাচাই করুন।

Reflection Questions

1. GKE Autopilot বনাম Standard — কখন কোনটি বেছে নেবেন?

2. Deployment-এ replica 2 রাখার সুবিধা কী? 1 রাখলে কী ঝুঁকি?

3. Liveness probe না থাকলে কী সমস্যা হতে পারে?

ল্যাব 19 • Lab Workbook

Lab 19 — Cloud Run Functions and API Gateway

Learning Objectives

- Cloud Run functions (2nd gen) দিয়ে event-driven function লিখতে পারবেন।
- API Gateway দিয়ে একাধিক backend-এর সামনে একটি API তৈরি করতে পারবেন।
- API key ও quota দিয়ে API সুরক্ষিত করতে পারবেন।

Prerequisites

- Lab 15 (Cloud Run), Lab 16 (Pub/Sub) সম্পন্ন। Node.js জানা।

Required Services

- Cloud Run functions, API Gateway, Pub/Sub।

Estimated Duration

৯০ মিনিট।

Cost-Risk Level

Low–Medium — Function invocation ও API Gateway billable, তবে ছোট test-এ খরচ কম।

Console Instructions

1. নিজের কম্পিউটারে ফোল্ডার তৈরি করুন `technova-order-function`। নিচের file লিখুন:

```
// package.json
{
  "name": "technova-order-function",
  "version": "1.0.0",
  "main": "index.js",
  "dependencies": { "@google-cloud/functions-framework": "^3.0.0" }
}
// index.js
const functions = require('@google-cloud/functions-framework');
functions.http('orderHandler', (req, res) => {
  console.log('Order received:', JSON.stringify(req.body));
  res.status(200).json({ received: true, orderId: req.body.orderId || 'unknown' });
});
```

1. Deploy করুন (নিচের command)। Region `asia-south1`, runtime Node.js 20, trigger HTTP, allow unauthenticated (ল্যাবের জন্য)।

2. Function URL-এ `curl` দিয়ে POST করুন: `{"orderId":"ORD-1002"}` — response `{"received":true}` আসা উচিত।
3. **Pub/Sub trigger:** একই function-এর একটি সংস্করণ Pub/Sub topic `technova-orders`-এর trigger দিয়ে deploy করুন (অথবা নতুন function তৈরি করুন)। Topic-এ message publish করলে function-এর log-এ message দেখা যাবে।
4. **API Gateway:** Console → **API Gateway** → **Create Gateway**। API নাম `technova-api-gw`। OpenAPI specification file তৈরি করুন (নিচে দেওয়া হয়েছে) যেখানে `/orders` path Lab 15-এর Cloud Run service-এ ও `/order-events` function-এ যায়।
5. Gateway তৈরি হলে তার URL-এ `/orders` কল করে backend response আসছে কি না দেখুন।
6. API key তৈরি করুন: Console → APIs & Services → Credentials → Create API Key। Gateway-এর OpenAPI-তে `security: api_key` যোগ করে key ছাড়া call 401/403 দেয় কি না পরীক্ষা করুন।

```
# openapi.yaml — Illustrative example (backend URL নিজের service URL দিয়ে বদলান)
swagger: '2.0'
info:
  title: TechNova API
  version: 1.0.0
schemes: [https]
produces: [application/json]
paths:
  /orders:
    get:
      operationId: getOrders
      x-google-backend:
        address: https://CHANGE-ME-CLOUD-RUN-URL
      responses:
        '200':
          description: OK
```

gcloud Commands

```
gcloud functions deploy technova-order-function --gen2 --runtime=nodejs20 --region=asia-south1
--source=. --entry-point=orderHandler --trigger-http --allow-unauthenticated
gcloud functions describe technova-order-function --gen2 --region=asia-south1 --
format="get(serviceConfig.uri)"
curl -X POST $(gcloud functions describe technova-order-function --gen2 --region=asia-south1
--format="get(serviceConfig.uri)") -H "Content-Type: application/json" -d
'{"orderId":"ORD-1002"}'
gcloud functions deploy technova-order-pubsub --gen2 --runtime=nodejs20 --region=asia-south1
--source=. --entry-point=orderHandler --trigger-topic=technova-orders
gcloud api-gateway apis create technova-api-gw --project=technova-lab
gcloud api-gateway api-configs create technova-api-config --api=technova-api-gw --openapi-
spec=openapi.yaml --project=technova-lab
gcloud api-gateway gateways create technova-gateway --api=technova-api-gw --api-
config=technova-api-config --location=asia-south1 --project=technova-lab
```

Expected Output

```
# Illustrative output
# $ curl -X POST FUNCTION_URL -d '{"orderId":"ORD-1002"}'
# {"received":true,"orderId":"ORD-1002"}
# Gateway URL-এ /orders কল করলে Cloud Run backend-এর response আসবে।
```

Verification

- Function URL-এ POST করে 200 response পাচ্ছেন কি না দেখুন।
- Pub/Sub-এ message publish করলে function-এর log-এ entry আসছে কি না দেখুন।
- Gateway URL-এ `/orders` কাজ করছে কি না যাচাই করুন।
- API key ছাড়া call ব্যর্থ হচ্ছে কি না পরীক্ষা করুন (key চালু থাকলে)।

Common Errors

Error	কারণ	সমাধান
Function deploy fails	Entry point বা runtime ভুল	`orderHandler` নাম ও Node.js 20 যাচাই করুন
Gateway 503	Backend URL ভুল বা backend বন্ধ	Cloud Run service URL ও status যাচাই করুন
API key not working	OpenAPI-তে security section নেই	OpenAPI file-এ api_key security যোগ করুন

Troubleshooting

- Function log দেখুন: `gcloud functions logs read technova-order-function --gen2 --region=asia-south1`।
- API Gateway-এর log Cloud Logging-এ দেখুন।

Security Considerations

- Function-এ unauthenticated access শুধু ল্যাবে; production-এ authentication চালু করুন।
- API key secret নয় — key-এর সঙ্গে quota ও IP restriction দিন।
- Function-এর service account-কে শুধু প্রয়োজনীয় permission দিন।

Cleanup and Deletion

- Function মুছুন: `gcloud functions delete technova-order-function --gen2 --region=asia-south1` ও `technova-order-pubsub` মুছুন।
- Gateway মুছুন: `gcloud api-gateway gateways delete technova-gateway --location=asia-south1`, তারপর api-config ও api মুছুন।
- API key মুছুন (Credentials থেকে)।

Reflection Questions

1. Cloud Run service বনাম Cloud Run functions — কখন কোনটি ব্যবহার করবেন?
2. API Gateway কী সমস্যার সমাধান করে? সরাসরি Cloud Run URL দিলে কী অসুবিধা?
3. API key, OAuth ও service account authentication-এর পার্থক্য কী?

ল্যাব 20 • Lab Workbook

Lab 20 — Terraform Infrastructure Provisioning

Learning Objectives

- Terraform দিয়ে VPC, subnet, firewall ও Cloud Run service-এর infrastructure-as-code লিখতে পারবেন।
- `terraform plan` ও `apply` দিয়ে resource তৈরি ও `destroy` দিয়ে মুছতে পারবেন।
- Remote state (GCS backend) ও variable ব্যবহার করতে পারবেন।

Prerequisites

- Lab 4–5, 11, 15 সম্পন্ন। Terraform ইনস্টল করা।

Required Services

- Terraform (local), Cloud Storage (state backend), Compute Engine, Cloud Run।

Estimated Duration

৯০–১২০ মিনিট।

Cost-Risk Level

Medium — Terraform যে resource তৈরি করে তার খরচ প্রযোজ্য; `destroy` দিয়ে সব মুছুন।

Console Instructions

1. State backend-এর জন্য একটি bucket তৈরি করুন: `technova-tf-state-CHANGE-ME-UNIQUE` (Lab 11-এর ধাপে, versioning On)।
2. নিজের কম্পিউটারে ফোল্ডার তৈরি করুন `technova-terraform`। নিচের file-গুলো লিখুন:

```
# main.tf — Illustrative example
terraform {
  required_providers {
    google = { source = "hashicorp/google", version = "~> 5.0" }
  }
  backend "gcs" {
    bucket = "technova-tf-state-CHANGE-ME-UNIQUE"
    prefix = "terraform/state"
  }
}
provider "google" {
  project = var.project_id
  region  = var.region
}
resource "google_compute_network" "technova_vpc" {
  name                    = "technova-tf-vpc"
```

```
  auto_create_subnetworks = false
}
resource "google_compute_subnetwork" "technova_web" {
  name                     = "technova-tf-web-subnet"
  ip_cidr_range            = "10.30.1.0/24"
  region                   = var.region
  network                  = google_compute_network.technova_vpc.id
  private_ip_google_access = true
}
resource "google_compute_firewall" "allow_http" {
  name    = "technova-tf-allow-http"
  network = google_compute_network.technova_vpc.name
  allow {
    protocol = "tcp"
    ports    = ["80", "443"]
  }
  source_ranges = ["0.0.0.0/0"]
  target_tags   = ["technova-web"]
}
# variables.tf
variable "project_id" { type = string }
variable "region" { type = string, default = "asia-south1" }
# terraform.tfvars (নিজের মান বসান — এই file Git-এ তুলবেন না)
project_id = "technova-lab"
region     = "asia-south1"
```

1. Terminal-এ চালান: `terraform init`, `terraform plan`, `terraform apply`।
2. Console-এ `technova-tf-vpc` তৈরি হয়েছে কি না দেখুন।
3. একটি পরিবর্তন করুন (যেমন firewall-এ port 22 যোগ করুন), আবার `plan` ও `apply` করুন — Terraform শুধু পরিবর্তনটুকু প্রয়োগ করবে।
4. `terraform state list` দিয়ে state-এ কী কী resource আছে দেখুন।

gcloud Commands

```
terraform --version
terraform init
terraform fmt
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
terraform state list
terraform output
terraform destroy
```

সতর্কতা — State file: Terraform state-এ resource-এর তথ্য থাকে, কখনো কখনো সংবেদনশীল মানও থাকতে পারে। State file local-এ রাখবেন না — GCS backend ব্যবহার করুন, bucket-এ versioning চালু রাখুন, আর `terraform.tfvars`-এ secret লিখবেন না।

Expected Output

```
# Illustrative output
# Plan: 3 to add, 0 to change, 0 to destroy.
# Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

Verification

- `terraform state list`-এ VPC, subnet ও firewall দেখা যাচ্ছে কি না দেখুন।
- Console-এ `technova-tf-vpc` ও subnet দেখুন।
- `terraform plan` আবার চালালে "No changes" আসছে কি না যাচাই করুন (idempotency)।

Common Errors

Error	কারণ	সমাধান
Backend bucket not found	State bucket তৈরি হয়নি	Bucket তৈরি করে `terraform init` আবার চালান
Resource already exists	Console/gcloud দিয়ে আগে তৈরি করা হয়েছে	`terraform import` করুন অথবা নাম বদলান
Permission denied on apply	Terraform চালানো account-এ role নেই	ADC account-এ প্রয়োজনীয় role দিন

Troubleshooting

- `terraform plan`-এর output মনোযোগ দিয়ে পড়ুন — কী তৈরি/পরিবর্তন/মুছবে তা আগেই দেখা যায়।
- State lock সমস্যায় GCS backend-এর lock যাচাই করুন; জোর করে unlock করবেন না।

Security Considerations

- State file-এ secret রাখবেন না; Secret Manager ব্যবহার করুন।
- `terraform.tfvars` ও `*.tfstate` Git-এ commit করবেন না — `.gitignore`-এ যোগ করুন।
- Terraform-এ service account ব্যবহার করুন CI/CD-তে, মানুষের credential নয়।

Cleanup and Deletion

- `terraform destroy` চালিয়ে সব Terraform resource মুছুন।
- State bucket খালি করে মুছুন (versioning থাকলে সব version মুছুন)।
- `terraform destroy` সফল হয়েছে কি না Console-এ যাচাই করুন।

Reflection Questions

1. Console-এ ক্লিক করে তৈরি করা বনাম Terraform — কোনটি repeatable, কেন?
2. Remote state কেন দরকার? Local state-এ কী সমস্যা?
3. `terraform plan` না দেখে `apply` করলে কী ঝুঁকি?

ল্যাব 21 • Lab Workbook

Lab 21 — Cloud Build and GitHub Actions CI/CD

Learning Objectives

- Cloud Build trigger দিয়ে Git push-এ image build ও Cloud Run deploy করতে পারবেন।
- GitHub Actions workflow দিয়ে একই CI/CD করতে পারবেন।
- Build log দেখে ব্যর্থ build-এর কারণ খুঁজতে পারবেন।

Prerequisites

- Lab 15, 17, 20 সম্পন্ন। GitHub account ও একটি test repository।

Required Services

- Cloud Build, Artifact Registry, Cloud Run, GitHub।

Estimated Duration

৯০–১২০ মিনিট।

Cost-Risk Level

Low–Medium — Cloud Build-এর free tier-এর মধ্যে ছোট build থাকার সম্ভাবনা বেশি; তবুও build সংখ্যা সীমিত রাখুন।

Console Instructions

1. Lab 15-এর `technova-api` code GitHub-এ একটি repository-তে push করুন (নাম: `technova-api`)।
2. Repository-তে `cloudbuild.yaml` file যোগ করুন:

```
# cloudbuild.yaml — Illustrative example
steps:
  - name: 'gcr.io/cloud-builders/docker'
    args: ['build', '-t', 'asia-south1-docker.pkg.dev/$PROJECT_ID/technova-images/technova-
api:$SHORT_SHA', '.']
  - name: 'gcr.io/cloud-builders/docker'
    args: ['push', 'asia-south1-docker.pkg.dev/$PROJECT_ID/technova-images/technova-api:
$SHORT_SHA']
  - name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
    entrypoint: gcloud
    args:
      - 'run'
      - 'deploy'
      - 'technova-api'
      - '--image=asia-south1-docker.pkg.dev/$PROJECT_ID/technova-images/technova-api:
$SHORT_SHA'
```

```
      - '--region=asia-south1'
      - '--allow-unauthenticated'
images:
  - 'asia-south1-docker.pkg.dev/$PROJECT_ID/technova-images/technova-api:$SHORT_SHA'
```

1. Console → **Cloud Build** → **Triggers** → **Create Trigger**। Event: Push to a branch, Source: GitHub repository `technova-api`, Branch: `main`। Configuration: Cloud Build configuration file (`cloudbuild.yaml`)। Service account: Cloud Build service account-কে Cloud Run Admin ও Artifact Registry Writer role দিন।
2. Code-এ ছোট পরিবর্তন করুন (যেমন response-এ version যোগ করুন), commit ও push করুন।
3. Console → Cloud Build → History-তে build চলছে কি না দেখুন। Build সফল হলে Cloud Run-এ নতুন revision এসেছে কি না যাচাই করুন।
4. **GitHub Actions:** Repository-তে `.github/workflows/deploy.yml` যোগ করুন — Google Cloud-এর `google-github-actions/auth` ও `deploy-cloudrun` action ব্যবহার করে একই deploy করুন। Workload Identity Federation ব্যবহার করুন (service account key নয়)।
5. GitHub Actions-এ workflow চালিয়ে deploy হচ্ছে কি না দেখুন।

> **সতর্কতা — GitHub secret:** Service account key GitHub secret-এ রাখা নিরাপত্তা ঝুঁকি। Workload Identity Federation (WIF) ব্যবহার করুন — এতে key ছাড়াই GitHub Actions GCP-তে authenticate করতে পারে। এই ল্যাবে WIF সেটআপের ধাপগুলো Console-এর নির্দেশনা অনুযায়ী করুন।

gcloud Commands

```
gcloud builds submit --config=cloudbuild.yaml --region=asia-south1 .
gcloud builds list --limit=5
gcloud builds log CHANGE-ME-BUILD-ID
gcloud run services describe technova-api --region=asia-south1 --
format="get(status.latestCreatedRevisionName)"
```

Expected Output

```
# Illustrative output
# Build SUCCESS — Cloud Run-এ নতুন revision তৈরি হয়েছে।
# $ curl CLOUD_RUN_URL/
# {"service":"technova-api","status":"ok","version":"v2"}
```

Verification

- Cloud Build History-তে build SUCCESS কি না দেখুন।
- Cloud Run-এ নতুন revision ও তার traffic 100% কি না যাচাই করুন।
- GitHub Actions-এর workflow run সফল হয়েছে কি না দেখুন।
- ইচ্ছাকৃতভাবে ভুল code push করে build ব্যর্থ হলে log-এ কারণ দেখা যায় কি না পরীক্ষা করুন, তারপর ঠিক করুন।

Common Errors

Error	কারণ	সমাধান
Build fails on docker push	Cloud Build SA-কে Writer role দেওয়া হয়নি	Artifact Registry Writer role দিন
Trigger not firing	Branch বা repository ভুল	Trigger-এর branch pattern যাচাই করুন
GitHub Actions auth fails	WIF সেটআপ ভুল	Workload Identity Pool ও provider যাচাই করুন

Troubleshooting

- Build log-এর প্রতিটি step-এর output দেখুন — কোন step-এ ব্যর্থ হয়েছে তা সেখানে লেখা থাকে।
- `gcloud builds submit` দিয়ে local-এ build পরীক্ষা করুন trigger-এর আগে।

Security Considerations

- Cloud Build service account-কে শুধু প্রয়োজনীয় role দিন।
- Build-এ secret ব্যবহার করলে Secret Manager ব্যবহার করুন, `cloudbuild.yaml`-এ plain text-এ লিখবেন না।
- Branch protection চালু করুন `main`-এ — সরাসরি push নয়, pull request দিয়ে merge করুন।

Cleanup and Deletion

- Trigger মুছুন: Console → Cloud Build → Triggers → Delete।
- GitHub workflow file মুছে ফেলুন বা repository archive করুন (যদি test repository হয়)।
- Cloud Run service ও image Lab 15-এর cleanup অনুযায়ী মুছুন (বা Lab 24-এর জন্য রেখে দিন — নোট রাখুন)।

Reflection Questions

1. Cloud Build trigger বনাম GitHub Actions — কখন কোনটি বেছে নেবেন?
2. Build-এ image tag হিসেবে `$SHORT_SHA` ব্যবহারের সুবিধা কী?
3. CI/CD-তে service account key ব্যবহারের বদলে WIF কেন নিরাপদ?

ল্যাব 22 • Lab Workbook

Lab 22 — Cloud Monitoring, Logging and Alerts

Learning Objectives

- Cloud Monitoring dashboard তৈরি করতে পারবেন।
- Log-based metric ও alert policy তৈরি করতে পারবেন।
- Uptime check ও notification channel সেট করতে পারবেন।
- Log Explorer দিয়ে error খুঁজতে পারবেন।

Prerequisites

- Lab 15 (Cloud Run service চলমান) সম্পন্ন।

Required Services

- Cloud Monitoring, Cloud Logging, Error Reporting।

Estimated Duration

৬০–৯০ মিনিট।

Cost-Risk Level

Low — Monitoring ও Logging-এর basic ব্যবহারে খরচ কম; log retention সীমিত রাখুন।

Console Instructions

1. Console → **Monitoring** → **Dashboards** → **Create Dashboard**। নাম: `technova-api-dashboard`।
2. Widget যোগ করুন: **Add Widget** → **Line chart**। Resource type: Cloud Run Revision, Metric: Request count। দ্বিতীয় chart: Request latencies (p95)। তৃতীয় chart: Container CPU utilization।
3. **Alerting** → **Create Policy:** Condition: Cloud Run revision-এর `5xx` error rate 5 মিনিটে 5%-এর বেশি হলে alert। Notification channel: Email (নিজের email) যোগ করুন। Documentation-এ runbook-এর সংক্ষিপ্ত নির্দেশনা লিখুন ("Cloud Run log দেখুন, revision যাচাই করুন, প্রয়োজনে rollback করুন")।
4. **Uptime check:** Monitoring → Uptime checks → Create। Title: `technova-api-uptime`, Resource type: URL, Hostname: Cloud Run service URL, Path: `/health`, Check every 1 minute। Alert চালু করুন।
5. **Log-based metric:** Logging → Log-based Metrics → Create Metric। নাম: `technova-api-errors`, Filter: `resource.type="cloud_run_revision" severity>=ERROR`।
6. Log Explorer-এ যান। Filter দিয়ে Cloud Run-এর error log খুঁজুন: `resource.type="cloud_run_revision" severity="ERROR"`।

7. ইচ্ছাকৃতভাবে একটি error তৈরি করুন (যেমন ভুল path-এ request করুন) — dashboard ও log-এ তা দেখা যাচ্ছে কি না পরীক্ষা করুন।

gcloud Commands

```
gcloud monitoring dashboards create --config-from-file=dashboard.json
gcloud monitoring channels create --display-name="TechNova Email" --type=email --channel-labels=email_address=CHANGE-ME-YOU@example.com
gcloud alpha monitoring policies create --config-from-file=alert-policy.yaml
gcloud logging read 'resource.type="cloud_run_revision" severity>=ERROR' --limit=10 --format=json
curl https://CHANGE-ME-CLOUD-RUN-URL/nonexistent-path
curl https://CHANGE-ME-CLOUD-RUN-URL/health
```

Alert-এর তিনটি অংশ: Condition (কখন alert হবে), Notification channel (কাকে জানাবে) ও Documentation (কী করতে হবে)। Documentation ছাড়া alert শুধু শব্দ — on-call ব্যক্তি কী করবে তা লিখে রাখুন।

Expected Output

```
# Illustrative output
# Dashboard-এ request count ও latency chart দেখা যাবে।
# Uptime check: UP (সবুজ)।
# Log Explorer-এ /nonexistent-path-এর 404 entry দেখা যাবে।
```

Verification

- Dashboard-এ তিনটি chart ডেটা দেখাচ্ছে কি না দেখুন।
- Alert policy enabled কি না: `gcloud alpha monitoring policies list`।
- Uptime check status UP কি না যাচাই করুন।
- Log-based metric-এ error count বাড়ছে কি না দেখুন (ভুল request-এর পর)।

Common Errors

Error	কারণ	সমাধান
Dashboard shows no data	Service-এ traffic নেই	কয়েকটি request পাঠান, তারপর refresh করুন
Alert never fires	Threshold খুব বেশি বা window ছোট	Threshold কমান; test-এর জন্য 1টি error-এ alert সেট করুন
Uptime check fails	Path ভুল বা service বন্ধ	`/health` path ও service URL যাচাই করুন

Troubleshooting

- Log Explorer-এ সময়ের range (time range) সঠিক কি না দেখুন — ভুল range-এ log দেখা যায় না।
- Alert-এর incident history দেখুন — alert fire হয়েছে কি না সেখানে লেখা থাকে।

Security Considerations

- Log-এ secret, password বা token লিখবেন না — log-এ যা যায় তা অনেকেই দেখতে পারে।
- Notification channel-এ শুধু প্রয়োজনীয় ব্যক্তিদের email দিন।
- Alert policy-তে sensitive তথ্য লিখবেন না।

Cleanup and Deletion

- Dashboard, alert policy ও uptime check মুছুন (অথবা Lab 24-এর জন্য রেখে দিন — নোট রাখুন)।
- Log-based metric মুছুন।
- Notification channel মুছুন।

Reflection Questions

1. Dashboard-এ কোন তিনটি metric সবচেয়ে গুরুত্বপূর্ণ — কেন?
2. Alert fatigue কী? কীভাবে এড়াবেন?
3. Log-based metric কখন ব্যবহার করবেন, কখন সরাসরি log search যথেষ্ট?

ল্যাব 23 • Lab Workbook

Lab 23 — Security Audit, Backup and Cost Analysis

Learning Objectives

- Security Command Center ও IAM recommender দিয়ে নিরাপত্তা audit করতে পারবেন।
- Backup ও restore পরীক্ষা করতে পারবেন।
- Cost breakdown দেখে খরচ কমানোর সুপারিশ দিতে পারবেন।
- একটি ছোট security ও cost রিপোর্ট লিখতে পারবেন।

Prerequisites

- Lab 1–22-এর বেশিরভাগ সম্পন্ন (audit করার মতো resource আছে)।

Required Services

- Security Command Center, IAM Recommender, Cloud Billing, Cloud SQL (backup পরীক্ষার জন্য)।

Estimated Duration

৯০ মিনিট।

Cost-Risk Level

Low–Medium — Audit-এ সরাসরি চার্জ কম; backup পরীক্ষার জন্য Cloud SQL instance লাগলে তার খরচ প্রযোজ্য।

Console Instructions

1. **IAM Audit:** Console → IAM & Admin → IAM। প্রতিটি principal-এর role দেখুন। `roles/owner` বা `roles/editor` কেউ পেয়েছে কি না দেখুন। **Recommender** (IAM → Recommendations) দেখুন — কোন role ব্যবহার হয়নি তা দেখাবে।
2. **Security Command Center:** Console → Security → Security Command Center → Findings। কোনো High/Critical finding আছে কি না দেখুন (যেমন public bucket, open firewall)। প্রতিটি finding-এর remediation ধাপ পড়ুন।
3. **Firewall Audit:** VPC network → Firewall। `0.0.0.0/0` থেকে SSH/RDP খোলা কোনো rule আছে কি না খুঁজুন। থাকলে তা Lab 5-এর নিয়ম অনুযায়ী সীমিত করুন।
4. **Backup Test:** Lab 13-এর Cloud SQL instance আবার তৈরি করুন (বা বিদ্যমানটি ব্যবহার করুন)। একটি test table-এ ডেটা লিখুন, তারপর **Backups** → **Create On-Demand Backup** করুন। Test ডেটা মুছে ফেলুন, তারপর backup থেকে restore করুন (অথবা clone তৈরি করে ডেটা ফেরত এসেছে কি না দেখুন)।
5. **Cost Analysis:** Console → Billing → Cost Table / Reports। গত ৩০ দিনের খরচ service অনুযায়ী দেখুন। সবচেয়ে বেশি খরচ কোন service-এ? কোন resource idle (চলছে কিন্তু ব্যবহার হচ্ছে না)?

6. **Idle Resource খুঁজুন:** Compute Engine → VM instances-এ stopped VM, unattached disk, unused external IP আছে কি না দেখুন। প্রতিটি idle resource-এর জন্য "রাখব / মুছব" সিদ্ধান্ত লিখুন।
7. এক পৃষ্ঠার রিপোর্ট লিখুন: (1) তিনটি security finding ও তার fix, (2) backup test-এর ফলাফল, (3) তিনটি cost-saving সুপারিশ।

gcloud Commands

```
gcloud projects get-iam-policy technova-lab --format=json > iam-policy.json
gcloud recommender recommendations list --project=technova-lab --location=global --
recommender=google.iam.policy.Recommender
gcloud compute firewall-rules list --filter="sourceRanges:0.0.0.0/0" --
format="table(name,allowed[].ports)"
gcloud compute disks list --filter="users:[]"
gcloud compute addresses list --filter="status:RESERVED"
gcloud billing budgets list --billing-account=CHANGE-ME-BILLING-ACCOUNT-ID
gcloud sql backups list --instance=technova-db
gcloud sql backups create --instance=technova-db --description="Lab 23 on-demand backup test"
```

Expected Output

```
# Illustrative output
# IAM policy-তে কোনো Owner/Editor binding থাকা উচিত নয় (নিজের account ছাড়া)।
# Idle disk list-এ unattached disk দেখা গেলে তা মুছতে হবে।
# Backup list-এ on-demand backup SUCCESS দেখা যাবে।
```

Verification

- IAM policy-তে অপ্রয়োজনীয় binding নেই তা নিশ্চিত করুন।
- Firewall-এ `0.0.0.0/0` থেকে SSH খোলা নেই তা যাচাই করুন।
- Backup থেকে ডেটা ফেরত এসেছে কি না পরীক্ষা করুন।
- Cost report-এ সবচেয়ে বড় তিনটি খরচের উৎস চিহ্নিত করুন।

Common Errors

Error	কারণ	সমাধান
SCC findings empty	SCC চালু হয়নি বা scan হয়নি	SCC enable করুন; কিছুক্ষণ অপেক্ষা করুন
Backup restore fails	Instance state বা disk space	Instance RUNNABLE কি না দেখুন; clone দিয়ে পরীক্ষা করুন
Cost data delayed	Billing data ২৪ ঘণ্টা পর্যন্ত দেরি হতে পারে	পরের দিন আবার দেখুন; budget alert-এর উপর নির্ভর করুন

Troubleshooting

- Recommender-এর সুপারিশ প্রয়োগ করার আগে তা কী পরিবর্তন করবে তা পড়ুন — ভুল role মুছে গেলে অ্যাপ বন্ধ হতে পারে।
- Restore পরীক্ষা production database-এ করবেন না — clone বা test instance ব্যবহার করুন।

Security Considerations

- Audit-এর ফলাফল (finding, IP, user email) public জায়গায় শেয়ার করবেন না।
- Backup-এ ডেটা থাকে — backup-এর access control যাচাই করুন।
- Unused service account ও key মুছে ফেলুন।

Cleanup and Deletion

- Test Cloud SQL instance মুছুন (Lab 13-এর cleanup অনুযায়ী)।
- Idle disk, unused IP ও stopped VM মুছুন।
- Audit রিপোর্টটি সংরক্ষণ করুন — Capstone-এ লাগবে।

Reflection Questions

1. Security audit কত ঘন ঘন করা উচিত? কেন?
2. Backup আছে কিন্তু restore পরীক্ষা করা হয়নি — এটি কি যথেষ্ট?
3. সবচেয়ে বড় খরচের উৎসটি কীভাবে কমাবেন — বন্ধ করে, ছোট করে, নাকি অন্য service ব্যবহার করে?

ল্যাব 24 • Lab Workbook

Lab 24 — Complete Production MERN Deployment

Learning Objectives

- Lab 1–23-এর সবকিছু একত্র করে একটি সম্পূর্ণ MERN অ্যাপ production-ধরনে deploy করতে পারবেন।
- Frontend (React/Next.js), Backend (Node.js/Express), Database (Cloud SQL / Firestore) ও Storage একসঙ্গে চালাতে পারবেন।
- CI/CD, monitoring, backup ও rollback সহ একটি সম্পূর্ণ deployment যাচাই করতে পারবেন।

Prerequisites

- Lab 1–23 সম্পন্ন। MERN অ্যাপের code (TechNova-র sample app — নিজের প্রজেক্ট, যেমন Bookora বা TeamFlow-এর সরল সংস্করণ, অথবা Lab 15-এর API + একটি React frontend)।

Required Services

- Cloud Run, Cloud SQL, Cloud Storage, Cloud Load Balancing, Cloud DNS, Secret Manager, Cloud Build, Cloud Monitoring, Artifact Registry।

Estimated Duration

১২০ মিনিট (বা দুই দিনে ভাগ করে)।

Cost-Risk Level

High — একাধিক billable service একসঙ্গে চলবে। সবচেয়ে ছোট tier ব্যবহার করুন, min-instances 0 রাখুন, ল্যাব শেষে সব মুছুন।

Console Instructions

Architecture:

User → Cloud DNS (technova.example) → HTTPS Load Balancer → Cloud Run (Frontend: React/Next.js) + Cloud Run (Backend: Node.js/Express API) → Cloud SQL (PostgreSQL, private IP) + Cloud Storage (uploads) + Firestore (tasks) → Cloud Monitoring / Logging / Alerts

1. **Database:** Lab 13-এর ধাপে Cloud SQL instance তৈরি করুন (`technova-prod-db`, private IP, automated backup On)। Database `technova_app`, user `technova_app_user`, password Secret Manager-এ।
2. **Storage:** Lab 11-এর ধাপে bucket তৈরি করুন (`technova-prod-uploads-CHANGE-ME-UNIQUE`, uniform access, public access prevention Enforced)।

3. **Backend:** Lab 15-এর ধাপে Node.js/Express API Cloud Run-এ deploy করুন (`technova-api`)। Environment variable: `DB_HOST` (Cloud SQL private IP বা Cloud SQL Auth Proxy), `DB_NAME=technova_app` । Secret: `DB_PASSWORD` Secret Manager থেকে। VPC Connector ব্যবহার করুন private IP-তে পৌঁছাতে (Serverless VPC Access)।
4. **Frontend:** React (বা Next.js) অ্যাপ build করে Cloud Run-এ deploy করুন (`technova-web`) — static build-এর জন্য nginx image অথবা Next.js standalone। API URL environment variable-এ দিন (hardcode করবেন না)।
5. **Load Balancing + DNS + HTTPS:** Lab 9 ও 12-এর ধাপে load balancer তৈরি করুন, দুটি Cloud Run service-কে backend হিসেবে (Serverless Network Endpoint Group ব্যবহার করুন), DNS A record ও Google-managed certificate যোগ করুন।
6. **CI/CD:** Lab 21-এর ধাপে Cloud Build trigger সেট করুন — `main` branch-এ push হলে backend ও frontend দুটোই build ও deploy হবে।
7. **Monitoring:** Lab 22-এর ধাপে dashboard, uptime check (`/health`) ও alert policy তৈরি করুন।
8. **End-to-end test:** ব্রাউজারে frontend খুলুন, একটি task তৈরি করুন, তা backend-এ গিয়ে database-এ সংরক্ষিত হচ্ছে কি না দেখুন। একটি file upload করে Cloud Storage-এ যাচ্ছে কি না দেখুন।
9. **Rollback test:** ইচ্ছাকৃতভাবে ভাঙা version deploy করুন, সমস্যা দেখুন, তারপর Cloud Run-এ আগের revision-এ traffic ফিরিয়ে দিন (rollback)।

gcloud Commands

```
gcloud run deploy technova-api --image=asia-south1-docker.pkg.dev/technova-lab/technova-images/technova-api:v2 --region=asia-south1 --service-account=technova-app-sa@technova-lab.iam.gserviceaccount.com --set-env-vars=NODE_ENV=production,DB_NAME=technova_app --set-secrets=DB_PASSWORD=technova-db-password:latest --vpc-connector=technova-vpc-connector --max-instances=2 --min-instances=0
gcloud run deploy technova-web --image=asia-south1-docker.pkg.dev/technova-lab/technova-images/technova-web:v1 --region=asia-south1 --allow-unauthenticated --max-instances=2 --min-instances=0
gcloud compute network-endpoint-groups create technova-api-neg --region=asia-south1 --network-endpoint-type=serverless --cloud-run-service=technova-api
gcloud run services update-traffic technova-api --region=asia-south1 --to-revisions=CHANGE-ME-PREVIOUS-REVISION=100
gcloud run services describe technova-api --region=asia-south1 --format="get(status.url)"
gcloud sql instances describe technova-prod-db --format="get(state)"
```

সতর্কতা — VPC Connector: Cloud Run থেকে private Cloud SQL-এ পৌঁছাতে Serverless VPC Access connector লাগে। Connector তৈরি করুন: `gcloud compute networks vpc-access connectors create technova-vpc-connector --network=technova-vpc --region=asia-south1 --range=10.40.0.0/28` । Connector-ও billable — ল্যাব শেষে মুছুন।

Expected Output

```
# Illustrative output
# Frontend: https://www.technova.example (বা Cloud Run URL) — TechNova অ্যাপের হোম পেজ।
```

```
# API: GET /health → {"status":"healthy"}
# API: POST /tasks → 201, task database-এ সংরক্ষিত।
# Rollback-এর পর আগের revision-এ traffic 100% ফিরে আসবে।
```

Verification

- Frontend, backend, database ও storage — প্রতিটি আলাদাভাবে কাজ করছে কি না যাচাই করুন।
- End-to-end: frontend থেকে task তৈরি → backend → database — পুরো পথটি কাজ করছে কি না দেখুন।
- Monitoring dashboard-এ request ও error দেখা যাচ্ছে কি না দেখুন।
- Rollback পরীক্ষা সফল হয়েছে কি না যাচাই করুন।
- Cost report দেখুন — এই deployment-এর দৈনিক খরচ কত? Pricing Calculator-এর অনুমানের সঙ্গে মিলছে কি না দেখুন।

Common Errors

Error	কারণ	সমাধান
Backend cannot reach Cloud SQL	VPC Connector নেই বা ভুল	Connector তৈরি করুন ও Cloud Run-এ attach করুন
Frontend cannot reach API	API URL ভুল বা CORS বন্ধ	API URL env যাচাই করুন; backend-এ CORS চালু করুন
502 from load balancer	Serverless NEG ভুল service-এ point করছে	NEG ও backend service যাচাই করুন

Troubleshooting

- প্রতিটি স্তর আলাদাভাবে পরীক্ষা করুন: প্রথমে database, তারপর backend, তারপর frontend, সবশেষে load balancer।
- Cloud Run log, Cloud SQL log ও Load Balancer log — তিনটি একসঙ্গে দেখুন।
- Rollback-এর জন্য revision-এর নাম নোট রাখুন: `gcloud run revisions list --service=technova-api --region=asia-south1`।

Security Considerations

- Database private IP-তে রাখুন; public IP দেবেন না।
- Secret Secret Manager-এ রাখুন; frontend-এ secret পাঠাবেন না।
- HTTPS বাধ্যতামূলক করুন; HTTP থেকে redirect করুন।
- Service account-কে least-privilege role দিন; Cloud Run-এর invoker permission শুধু প্রয়োজনীয় principal-কে দিন।

Cleanup and Deletion

- Load balancer, NEG, certificate ও DNS record মুছুন।
- Cloud Run service দুটি মুছুন: `gcloud run services delete technova-api technova-web --region=asia-south1`।
- VPC Connector মুছুন।
- Cloud SQL instance মুছুন।
- Bucket খালি করে মুছুন।
- Artifact Registry image ও Cloud Build trigger মুছুন।
- সবশেষে `gcloud compute instances list`, `gcloud sql instances list`, `gcloud run services list` দিয়ে নিশ্চিত করুন — কোনো billable resource চালু নেই।
- Billing → Cost Table-এ শেষ খরচ দেখে নিন।

Reflection Questions

1. এই deployment-এর কোন অংশটি সবচেয়ে ভঙ্গুর (fragile)? কীভাবে শক্তিশালী করবেন?
2. Production-এ এই অ্যাপ চালাতে মাসে কত খরচ হতে পারে — Pricing Calculator দিয়ে অনুমান করুন।
3. আপনি কী কী শিখলেন যা Lab 1-এ জানতেন না? পরের প্রজেক্টে কী ভিন্নভাবে করবেন?

Assignment 1 • Lab Workbook

Assignment 1 — Design a Custom VPC for TechNova

TechNova-র জন্য একটি custom VPC নকশা করুন। Requirement: web, application ও database — তিনটি আলাদা স্তর; দুটি region (`asia-south1` ও `asia-southeast1`); database স্তরে কোনো public IP থাকবে না।

কাজ

- প্রতিটি region-এ তিনটি subnet-এর CIDR পরিকল্পনা করুন (non-overlapping, ভবিষ্যতের বৃদ্ধির জন্য জায়গা রেখে)। একটি টেবিলে subnet, CIDR, region ও উদ্দেশ্য লিখুন।
- Firewall rule-এর তালিকা লিখুন: কোন rule, কোন source, কোন target tag, কোন port — এবং প্রতিটি rule-এর যুক্তি।
- Cloud NAT ও Private Google Access কোথায় লাগবে তা চিহ্নিত করুন।
- একটি network diagram আঁকুন (হাতে বা diagram tool-এ) এবং ব্যাখ্যা করুন কেন এই নকশা নিরাপদ।

Deliverable

একটি নকশা নথি (২–৩ পৃষ্ঠা): CIDR টেবিল, firewall rule তালিকা, diagram ও নিরাপত্তা যুক্তি।

মূল্যায়নের মানদণ্ড

- CIDR overlap নেই ও ভবিষ্যতের বৃদ্ধির জায়গা আছে।
- Database স্তর সম্পূর্ণ private।
- Firewall rule least-privilege অনুযায়ী লেখা।
- Diagram ও ব্যাখ্যা স্পষ্ট।

Assignment 2 • Lab Workbook

Assignment 2 — Write an IAM Policy for a Small Team

TechNova-র ছোট দল: 1 জন Tech Lead, 2 জন Developer, 1 জন QA, 1 জন Finance (billing দেখে)। প্রত্যেকের জন্য IAM policy লিখুন।

কাজ

- প্রতিটি ভূমিকার জন্য প্রয়োজনীয় predefined role-এর তালিকা করুন। কাউকে Owner/Editor দেবেন না।
- একটি service account-এর জন্য policy লিখুন — Cloud Run অ্যাপটি Cloud SQL ও Cloud Storage ব্যবহার করবে।
- কোন role কেন দেওয়া হয়েছে, আর কোন role কেন দেওয়া হয়নি — প্রতিটির যুক্তি লিখুন।
- `gcloud projects add-iam-policy-binding` command দিয়ে policy প্রয়োগের ধাপ লিখুন।
- একজন Developer কোম্পানি ছাড়লে কী কী করতে হবে — একটি offboarding checklist লিখুন।

Deliverable

IAM policy নথি: ভূমিকা-ভিত্তিক role টেবিল, service account policy, যুক্তি ও offboarding checklist।

মূল্যায়নের মানদণ্ড

- Least privilege প্রতিটি ভূমিকায় মানা হয়েছে।
- Finance-কে শুধু billing দেখার role দেওয়া হয়েছে, resource পরিবর্তনের role নয়।
- Service account-এর role শুধু প্রয়োজনীয়।

Assignment 3 • Lab Workbook

Assignment 3 — Build a Cloud Run Deployment Plan

TechNova-র Node.js/Express API-টি Cloud Run-এ deploy করার সম্পূর্ণ পরিকল্পনা লিখুন।

কাজ

- Dockerfile লিখুন (multi-stage build, non-root user, ছোট base image)।
- Artifact Registry repository, image tag কৌশল (digest বনাম tag) ও cleanup policy লিখুন।
- Cloud Run service-এর configuration লিখুন: region, min/max instances, CPU/memory, concurrency, timeout, service account, environment variable ও Secret Manager-এর ব্যবহার।
- Deploy ধাপ, verification ধাপ ও rollback ধাপ লিখুন।
- Health check endpoint (`/health`) কী কী যাচাই করবে তা লিখুন।

Deliverable

Deployment পরিকল্পনা নথি + Dockerfile + `gcloud run deploy` command।

মূল্যায়নের মানদণ্ড

- Dockerfile নিরাপদ ও ছোট।
- Secret কোথাও hardcode করা হয়নি।
- Rollback ধাপ স্পষ্ট।

Assignment 4 • Lab Workbook

Assignment 4 — Design a GKE Platform for TechNova

TechNova-র পাঁচটি microservice GKE-তে চালানোর প্ল্যাটফর্ম নকশা করুন।

কাজ

- Autopilot বনাম Standard cluster — কোনটি বেছে নেবেন, যুক্তি দিন।
- Namespace নকশা করুন (যেমন `production`, `staging`) ও প্রতিটি namespace-এর resource quota লিখুন।
- Deployment, Service, Ingress ও Horizontal Pod Autoscaler-এর উদাহরণ manifest লিখুন (একটি service-এর জন্য)।
- Workload Identity দিয়ে service account-এর সংযোগ ব্যাখ্যা করুন।
- Node pool, private cluster ও network policy-এর সিদ্ধান্ত লিখুন।

Deliverable

প্ল্যাটফর্ম নকশা নথি + একটি service-এর সম্পূর্ণ Kubernetes manifest।

মূল্যায়নের মানদণ্ড

- Autopilot/Standard সিদ্ধান্তের যুক্তি স্পষ্ট।
- Resource request/limit ও HPA সঠিকভাবে সেট করা।
- Workload Identity ও private cluster বিবেচনা করা হয়েছে।

Assignment 5 • Lab Workbook

Assignment 5 — Cost Analysis and FinOps Report

TechNova-র গত মাসের GCP খরচ বিশ্লেষণ করে একটি FinOps রিপোর্ট লিখুন। Lab 23-এর cost analysis ধাপ ব্যবহার করুন, অথবা একটি কাল্পনিক খরচের টেবিল তৈরি করুন (service অনুযায়ী)।

কাজ

- Service অনুযায়ী খরচের breakdown টেবিল তৈরি করুন (Compute Engine, Cloud SQL, Cloud Run, Storage, Networking, অন্যান্য)।
- সবচেয়ে বেশি খরচের তিনটি উৎস চিহ্নিত করুন ও প্রতিটির জন্য খরচ কমানোর উপায় লিখুন (যেমন idle resource মুছে ফেলা, ছোট machine type, committed use discount, lifecycle policy)।
- Google Cloud Pricing Calculator ব্যবহার করে একটি ছোট deployment-এর মাসিক অনুমান তৈরি করুন।
- Billing budget ও alert-এর সুপারিশ লিখুন (threshold ও notification)।
- Labeling কৌশল লিখুন — কোন label দিয়ে খরচ ভাগ করা যাবে (যেমন `env`, `team`, `cost-center`)।

Deliverable

FinOps রিপোর্ট (২–৩ পৃষ্ঠা): breakdown টেবিল, তিনটি সাশ্রয় সুপারিশ, Pricing Calculator অনুমান ও budget সুপারিশ।

মূল্যায়নের মানদণ্ড

- খরচের উৎস সঠিকভাবে চিহ্নিত হয়েছে।
- সুপারিশগুলো বাস্তবসম্মত ও পরিমাপযোগ্য।
- কোনো price hardcode করা হয়নি — Calculator-এর উল্লেখ আছে।

Assignment 6 • Lab Workbook

Assignment 6 — Disaster Recovery Plan for Cloud SQL and Storage

TechNova-র database ও file storage-এর জন্য একটি disaster recovery (DR) পরিকল্পনা লিখুন।

কাজ

- RPO (Recovery Point Objective) ও RTO (Recovery Time Objective) নির্ধারণ করুন — database ও storage-এর জন্য আলাদাভাবে।
- Cloud SQL-এর automated backup, point-in-time recovery ও cross-region replica-এর পরিকল্পনা লিখুন।
- Cloud Storage-এর versioning, lifecycle ও cross-region replication (dual-region/multi-region bucket) পরিকল্পনা লিখুন।
- Restore পরীক্ষার ধাপ লিখুন — কত ঘন ঘন পরীক্ষা করবেন, কীভাবে করবেন, সফলতার মানদণ্ড কী।
- একটি incident scenario লিখুন: "প্রধান region-এর database নষ্ট হয়ে গেছে" — ধাপে ধাপে কী করবেন।

Deliverable

DR পরিকল্পনা নথি: RPO/RTO টেবিল, backup কৌশল, restore পরীক্ষার ধাপ ও incident scenario।

মূল্যায়নের মানদণ্ড

- RPO/RTO বাস্তবসম্মত ও যুক্তিসঙ্গত।
- Restore পরীক্ষার ধাপ আছে — শুধু backup থাকাই যথেষ্ট নয়।
- Cross-region কৌশল বিবেচনা করা হয়েছে।

Assignment 7 • Lab Workbook

Assignment 7 — Design a CI/CD Pipeline with Cloud Build

TechNova-র MERN অ্যাপের জন্য একটি CI/CD pipeline নকশা করুন — code push থেকে production deploy পর্যন্ত।

কাজ

- Pipeline-এর ধাপগুলো লিখুন: test, build, scan, deploy to staging, approval, deploy to production।
- `cloudbuild.yaml` লিখুন (build, push, deploy ধাপ সহ)।
- Branch কৌশল লিখুন: `main`, `develop`, feature branch — কোন branch থেকে কোথায় deploy হবে।
- Image vulnerability scan ও approval ধাপ কোথায় থাকবে তা দেখান।
- Deploy ব্যর্থ হলে rollback ধাপ লিখুন।

Deliverable

Pipeline নকশা নথি + `cloudbuild.yaml` + branch কৌশল diagram।

মূল্যায়নের মানদণ্ড

- Staging ও production আলাদা।
- Test ও scan ধাপ deploy-এর আগে আছে।
- Rollback ধাপ স্পষ্ট।

Assignment 8 • Lab Workbook

Assignment 8 — Monitoring, Logging and Alerting Design

TechNova-র production অ্যাপের জন্য observability নকশা করুন।

কাজ

- পাঁচটি গুরুত্বপূর্ণ metric নির্বাচন করুন (যেমন request rate, error rate, latency p95, CPU, database connection) ও প্রতিটির alert threshold লিখুন।
- একটি dashboard-এর layout আঁকুন — কোন chart কোথায় থাকবে।
- তিনটি alert policy লিখুন: condition, notification channel, documentation (runbook-এর সংক্ষিপ্ত ধাপ)।
- Uptime check-এর পরিকল্পনা লিখুন (কোন URL, কত ঘন ঘন, কোন region থেকে)।
- Log-এ কী কী লিখবেন আর কী লিখবেন না (secret, PII) — একটি নিয়মের তালিকা করুন।

Deliverable

Observability নকশা নথি: metric টেবিল, dashboard layout, তিনটি alert policy ও logging নিয়ম।

মূল্যায়নের মানদণ্ড

- Alert threshold বাস্তবসম্মত (খুব কম বা খুব বেশি নয়)।
- প্রতিটি alert-এ runbook আছে।
- Log-এ secret না লেখার নিয়ম আছে।

Assignment 9 • Lab Workbook

Assignment 9 — Secure a Public-Facing Application

TechNova-র public web অ্যাপটির নিরাপত্তা পর্যালোচনা করে একটি hardening পরিকল্পনা লিখুন।

কাজ

- অ্যাপের attack surface তালিকা করুন (load balancer, Cloud Run, database, storage, DNS)।
- প্রতিটি স্তরে নিরাপত্তা নিয়ন্ত্রণ লিখুন: HTTPS, firewall, IAM, Secret Manager, Cloud Armor, VPC Service Controls (ধারণা)।
- একটি security checklist তৈরি করুন (deploy-এর আগে যাচাই করার জন্য)।
- একটি incident scenario লিখুন: "সন্দেহজনক login চেষ্টা বেড়ে গেছে" — কী করবেন।
- Security Command Center-এর finding-গুলো কীভাবে দেখবেন ও ঠিক করবেন তা লিখুন।

Deliverable

Security hardening নথি: attack surface টেবিল, স্তরভিত্তিক নিয়ন্ত্রণ, checklist ও incident scenario।

মূল্যায়নের মানদণ্ড

- প্রতিটি স্তরে নিয়ন্ত্রণ আছে।
- Least privilege ও private networking বিবেচনা করা হয়েছে।
- Checklist বাস্তবসম্মত।

Assignment 10 • Lab Workbook

Assignment 10 — Migrate a VM Workload to Cloud Run

TechNova-র একটি পুরোনো VM-এ চলা Node.js অ্যাপ Cloud Run-এ migrate করার পরিকল্পনা লিখুন।

কাজ

- বর্তমান VM অ্যাপের বিবরণ লিখুন (কী করে, কোন port, কোন ডেটা ব্যবহার করে, কোন file লেখে)।
- Cloud Run-এ migrate করতে কী কী পরিবর্তন লাগবে তা তালিকা করুন (stateless করা, file storage-কে Cloud Storage-এ নেওয়া, environment variable, health check, PORT)।
- Migration-এর ধাপ লিখুন: প্রথমে staging-এ deploy, তারপর traffic ধীরে ধীরে সরানো (10% → 50% → 100%)।
- Rollback পরিকল্পনা লিখুন — সমস্যা হলে VM-এ ফিরে যাবেন কীভাবে।
- Migration-এর পর খরচ ও performance-এর তুলনা কীভাবে করবেন তা লিখুন।

Deliverable

Migration পরিকল্পনা নথি: পরিবর্তনের তালিকা, ধাপ, traffic migration ও rollback।

মূল্যায়নের মানদণ্ড

- Stateless করার পরিবর্তনগুলো চিহ্নিত হয়েছে।
- ধীরে ধীরে traffic সরানোর পরিকল্পনা আছে।
- Rollback ধাপ স্পষ্ট।

Assignment 11 • Lab Workbook

Assignment 11 — Design an Event-Driven System with Pub/Sub

TechNova-র order প্রক্রিয়া event-driven করার নকশা করুন: order তৈরি হলে inventory, payment ও notification — তিনটি কাজ আলাদাভাবে হবে।

কাজ

- Topic ও subscription-এর নকশা আঁকুন (কোন topic, কোন subscriber, push না pull)।
- Message-এর format (JSON schema) লিখুন — কোন field থাকবে, কোনটি থাকবে না (secret/PII নয়)।
- Retry, dead-letter topic ও idempotency (duplicate message সামলানো) পরিকল্পনা লিখুন।
- একটি failure scenario লিখুন: payment service বন্ধ থাকলে কী হবে, message কোথায় যাবে, কীভাবে পুনরুদ্ধার হবে।

Deliverable

Event-driven নকশা নথি: topic/subscription diagram, message schema, retry/DLQ পরিকল্পনা ও failure scenario।

মূল্যায়নের মানদণ্ড

- প্রতিটি subscriber-এর কাজ স্পষ্ট।
- DLQ ও retry পরিকল্পনা আছে।
- Duplicate message সামলানোর উপায় লেখা আছে।

Assignment 12 • Lab Workbook

Assignment 12 — Terraform Module for a Reusable VPC

TechNova-র প্রতিটি নতুন প্রজেক্টে একই VPC নকশা ব্যবহার করার জন্য একটি reusable Terraform module লিখুন।

কাজ

- Module-এর input variable লিখুন (project ID, region, CIDR range, environment)।
- Module-এর resource লিখুন: VPC, দুটি subnet, firewall rule, Cloud NAT (ঐচ্ছিক)।
- Module-এর output লিখুন (VPC নাম, subnet নাম, CIDR)।
- Module ব্যবহার করে একটি উদাহরণ `main.tf` লিখুন (staging ও production — দুটি environment)।
- State backend ও `.gitignore` নিয়ম লিখুন।

Deliverable

Terraform module code + উদাহরণ ব্যবহার + README (কীভাবে ব্যবহার করবেন)।

মূল্যায়নের মানদণ্ড

- Module variable দিয়ে পুনরায় ব্যবহারযোগ্য।
- Secret কোথাও hardcode করা হয়নি।
- Staging ও production আলাদা state-এ আছে।

Assignment 13 • Lab Workbook

Assignment 13 — Load Testing and Autoscaling Analysis

TechNova-র API-র load test পরিকল্পনা করুন ও autoscaling বিশ্লেষণ করুন।

কাজ

- Load test-এর পরিকল্পনা লিখুন: কত concurrent user, কত সময়, কোন endpoint, সফলতার মানদণ্ড (p95 latency, error rate)।
- Cloud Run-এর autoscaling সেটিং লিখুন (min/max instances, concurrency) ও কেন এই মান বেছে নিলেন তা ব্যাখ্যা করুন।
- MIG-এর autoscaling-এর সঙ্গে Cloud Run-এর autoscaling-এর পার্থক্য লিখুন।
- Load test-এর ফলাফল থেকে তিনটি সুপারিশ লিখুন (যেমন instance বাড়ানো, cache যোগ করা, database connection বাড়ানো)।
- Load test করার সময় খরচ ও production-এর উপর প্রভাব কীভাবে নিয়ন্ত্রণ করবেন তা লিখুন (staging-এ test, সীমা দেওয়া)।

Deliverable

Load test পরিকল্পনা ও autoscaling বিশ্লেষণ নথি।

মূল্যায়নের মানদণ্ড

- সফলতার মানদণ্ড পরিমাপযোগ্য।
- Autoscaling সেটিং-এর যুক্তি স্পষ্ট।
- Production-এ load test না করার সতর্কতা আছে।

Assignment 14 • Lab Workbook

Assignment 14 — Multi-Environment Strategy (Dev, Staging, Production)

TechNova-র জন্য dev, staging ও production — তিনটি environment-এর কৌশল লিখুন।

কাজ

- প্রতিটি environment-এর জন্য প্রজেক্ট কৌশল লিখুন (একটি প্রজেক্টে তিনটি environment, নাকি আলাদা প্রজেক্ট — যুক্তি দিন)।
- প্রতিটি environment-এর resource আকার লিখুন (dev ছোট, production বড় — কেন)।
- Code কীভাবে dev → staging → production যাবে তা লিখুন (branch, CI/CD, approval)।
- প্রতিটি environment-এর ডেটা কৌশল লিখুন (production ডেটা dev-এ copy করা যাবে কি না — PII বিবেচনা করুন)।
- Environment-ভিত্তিক খরচের পার্থক্য ব্যাখ্যা করুন।

Deliverable

Multi-environment কৌশল নথি: প্রজেক্ট নকশা, promotion ধাপ, ডেটা নিয়ম ও খরচ তুলনা।

মূল্যায়নের মানদণ্ড

- আলাদা প্রজেক্ট বনাম এক প্রজেক্টের যুক্তি স্পষ্ট।
- Production ডেটা dev-এ নেওয়ার ঝুঁকি বিবেচনা করা হয়েছে।
- Promotion ধাপে approval আছে।

Assignment 15 • Lab Workbook

Assignment 15 — Write an Operational Runbook

TechNova-র production অ্যাপের জন্য একটি operational runbook লিখুন — on-call ব্যক্তি রাত ৩টায় alert পেলে কী করবে।

কাজ

- তিনটি সাধারণ incident-এর runbook লিখুন: (1) API 5xx error বেড়ে গেছে, (2) database connection শেষ হয়ে গেছে, (3) Cloud Run service-এ traffic হঠাৎ বেড়ে খরচ বাড়ছে।
- প্রতিটি runbook-এ লিখুন: লক্ষণ (symptom), যাচাই করার ধাপ, সম্ভাব্য কারণ, সমাধান, কখন escalate করবেন, rollback ধাপ।
- Runbook-এর শুরুতে লিখুন: গুরুত্বপূর্ণ link (dashboard, log, status page), যোগাযোগের তালিকা, ও service-এর architecture সংক্ষেপ।
- Runbook কীভাবে আপডেট করবেন (প্রতিটি incident-এর পর) তা লিখুন।

Deliverable

Operational runbook (৩–৪ পৃষ্ঠা): তিনটি incident runbook + পরিচিতি বিভাগ।

মূল্যায়নের মানদণ্ড

- প্রতিটি runbook ধাপে ধাপে অনুসরণযোগ্য।
- Escalation নিয়ম স্পষ্ট।
- Rollback ধাপ আছে।

Project 01 • Lab Workbook

Project 01 — Secure GCP Network & Compute Foundation

TechNova-র জন্য একটি নিরাপদ GCP network ও compute ভিত্তি তৈরি করুন — যার উপর পরের সব অ্যাপ চলবে। এই প্রজেক্টে Lab 1–10-এর সবকিছু একত্র হয়।

Requirements

- Custom VPC (`technova-vpc`) — regional subnet: web (`10.10.1.0/24`) ও database (`10.10.2.0/24`), `asia-south1` region-এ।
- Firewall rules — least privilege: SSH শুধু নিজের IP থেকে, HTTP/HTTPS শুধু web tag-এ, internal traffic শুধু VPC-র ভেতরে।
- Cloud NAT ও Cloud Router — private VM-এর outbound internet।
- IAM ও service accounts — `technova-app-sa` শুধু প্রয়োজনীয় role সহ; কাউকে Owner/Editor নয়।
- Compute Engine — instance template, Managed Instance Group (min 1, max 3), autoscaling (CPU 60%), autohealing।
- Load Balancing — external Application Load Balancer, health check, backend service।
- Network documentation — CIDR টেবিল, firewall rule তালিকা, diagram।
- Billing budget ও alert সেট করা থাকতে হবে।

Architecture

User → External Application Load Balancer (HTTP/HTTPS) → Managed Instance Group (technova-web-mig, 1–3 VMs, asia-south1) → technova-vpc: web subnet 10.10.1.0/24 → Cloud NAT → Internet (outbound only) | Database subnet 10.10.2.0/24 (private, no public IP) | IAM: technova-app-sa (least privilege) | Billing budget + alerts

Implementation Stages

1. **Stage 1 — Foundation:** প্রজেক্ট, billing budget, IAM ও service account (Lab 1–3)।
2. **Stage 2 — Networking:** VPC, subnet, firewall, NAT, Private Google Access (Lab 4–6)।
3. **Stage 3 — Compute:** Instance template, MIG, autoscaling, autohealing (Lab 7, 10)।
4. **Stage 4 — Load Balancing:** Health check, backend service, load balancer, traffic পরীক্ষা (Lab 9)।
5. **Stage 5 — Hardening & Documentation:** Firewall audit, IAM audit, network documentation লেখা।

Verification

- Load balancer IP-তে বারবার request করে দুটি VM-এর response পাওয়া যাচ্ছে কি না দেখুন।

- একটি VM বন্ধ করলে traffic সুস্থ VM-এ সরে যাচ্ছে কি না পরীক্ষা করুন।
- CPU load বাড়ালে MIG-এ VM সংখ্যা বাড়ছে কি না দেখুন।
- Database subnet-এর VM-এ public IP নেই তা যাচাই করুন।
- SSH শুধু নিজের IP থেকে কাজ করছে কি না পরীক্ষা করুন।

Security Review

- IAM policy-তে কোনো Owner/Editor binding নেই কি না যাচাই করুন।
- Firewall-এ `0.0.0.0/0` থেকে SSH খোলা নেই কি না দেখুন।
- Service account key তৈরি করা হয়নি কি না যাচাই করুন।
- Private VM-এ public IP নেই কি না দেখুন।

Cost Review

- Pricing Calculator দিয়ে MIG (1–3 × e2-micro) + load balancer + NAT-এর মাসিক অনুমান তৈরি করুন।
- Min instances 1 রাখার খরচ বনাম availability-এর trade-off লিখুন।
- Idle resource আছে কি না যাচাই করুন।

Final Deliverables

- চালু থাকা infrastructure (load balancer IP, MIG, VPC) — অথবা Terraform code (Lab 20-এর ধাপে)।
- Network documentation: CIDR টেবিল, firewall rule তালিকা, architecture diagram।
- Security review checklist (সম্পন্ন)।
- Cost estimate (Pricing Calculator-এর উল্লেখ সহ)।
- Cleanup পরিকল্পনা — কোন resource কখন মুছবেন।

> **মনে রাখুন:** প্রজেক্ট শেষে billable resource (MIG, load balancer, NAT) মুছে ফেলুন, অথবা Project 02-এর ভিত্তি হিসেবে রাখলে তা নোট করে রাখুন ও billing budget যাচাই করুন।

Project 02 • Lab Workbook

Project 02 — Production MERN Application on GCP

TechNova-র MERN অ্যাপ (React/Next.js frontend, Node.js/Express backend) production-ধরনে GCP-তে deploy করুন। এই প্রজেক্টে Lab 11–17, 19, 21–22-এর সবকিছু একত্র হয়।

Requirements

- Frontend — React বা Next.js, Cloud Run-এ deploy (`technova-web`)।
- Backend — Node.js + Express API, Cloud Run-এ deploy (`technova-api`), `/health` endpoint সহ।
- Database — Cloud SQL (PostgreSQL), private IP, automated backup On।
- Storage — Cloud Storage bucket (uploads), uniform access, public access prevention Enforced।
- Load Balancing, Cloud DNS ও HTTPS — Google-managed certificate, HTTP থেকে HTTPS redirect।
- CI/CD — Cloud Build trigger বা GitHub Actions; `main` branch-এ push হলে deploy হবে।
- Secret Manager — database password ও API key Secret Manager-এ; code-এ কোনো secret নয়।
- Monitoring ও Logging — dashboard, uptime check (`/health`), alert policy।

Architecture

User → Cloud DNS (technova.example) → HTTPS Load Balancer → Cloud Run technova-web (Frontend) + Cloud Run technova-api (Backend) → Cloud SQL PostgreSQL (private IP) + Cloud Storage (uploads) | Secret Manager (DB password) | Cloud Build CI/CD ← GitHub | Cloud Monitoring + Logging + Alerts

Implementation Stages

1. **Stage 1 — Data:** Cloud SQL, database, user, Secret Manager, Storage bucket (Lab 11, 13)।
2. **Stage 2 — Backend:** API containerize, Artifact Registry-তে push, Cloud Run-এ deploy, database সংযোগ পরীক্ষা (Lab 15, 17)।
3. **Stage 3 — Frontend:** React/Next.js build, Cloud Run-এ deploy, API URL environment variable-এ (Lab 15)।
4. **Stage 4 — Networking:** Load balancer, DNS, HTTPS certificate, CDN (Lab 9, 12)।
5. **Stage 5 — CI/CD:** Cloud Build trigger, staging deploy, production deploy (Lab 21)।
6. **Stage 6 — Observability:** Dashboard, uptime check, alert, log-based metric (Lab 22)।

Verification

- End-to-end: frontend থেকে task তৈরি → backend → database — পুরো পথ কাজ করছে কি না দেখুন।
- File upload Cloud Storage-এ যাচ্ছে কি না পরীক্ষা করুন।
- HTTPS-এ site খুলছে কি না ও HTTP redirect হচ্ছে কি না দেখুন।

- CI/CD: ছোট পরিবর্তন push করে নতুন revision deploy হচ্ছে কি না দেখুন।
- Rollback: আগের revision-এ traffic ফিরিয়ে দিন।

Security Review

- Database public IP বন্ধ কি না যাচাই করুন।
- Secret code-এ বা log-এ নেই কি না দেখুন।
- Cloud Run-এর invoker permission শুধু প্রয়োজনীয় principal-কে দেওয়া হয়েছে কি না দেখুন।
- Bucket public নয় কি না যাচাই করুন।

Cost Review

- Pricing Calculator দিয়ে পুরো deployment-এর মাসিক অনুমান তৈরি করুন।
- Min-instances 0 রাখার সুবিধা ও cold start-এর trade-off লিখুন।
- সবচেয়ে বেশি খরচের service চিহ্নিত করুন।

Final Deliverables

- চালু থাকা MERN অ্যাপ (URL সহ) — অথবা deploy ধাপের সম্পূর্ণ নথি।
- Source code repository (GitHub link) + `cloudbuild.yaml`।
- Architecture diagram ও deployment নথি।
- Security review checklist ও cost estimate।
- Rollback ও cleanup পরিকল্পনা।

Project 03 • Lab Workbook

Project 03 — TechNova GCP Cloud Platform

TechNova-র সম্পূর্ণ GCP cloud platform তৈরি করুন — governance থেকে observability পর্যন্ত। এটি এই বইয়ের সবচেয়ে বড় প্রজেক্ট; Project 01 ও 02-এর উপর ভিত্তি করে তৈরি।

Requirements

- Organization ও project governance — Folder কাঠামো, project নামকরণ, label কৌশল, billing budget।
- Secure networking — custom VPC, private subnet, firewall (least privilege), Cloud NAT, Private Service Connect, VPC Service Controls ধারণা।
- GKE platform — Autopilot বা Standard cluster, namespace, Workload Identity, HPA, Ingress।
- Managed databases ও storage — Cloud SQL (private IP, backup), Firestore, Cloud Storage (lifecycle, versioning)।
- Centralized observability — Monitoring dashboard, Logging, alert policy, uptime check, log-based metric।
- Security controls — IAM audit, Security Command Center finding পর্যালোচনা, Secret Manager, Cloud Armor (ঐচ্ছিক)।
- Backup ও DR — Cloud SQL backup, restore পরীক্ষা, RPO/RTO নথি।
- FinOps — cost report, budget alert, idle resource পর্যালোচনা, Pricing Calculator অনুমান।

Architecture

Organization technova.example → Folder production / staging → Projects (network, app, data) → VPC (web/app/db subnet, NAT, PSC) → GKE (microservices, Workload Identity) + Cloud Run (API) → Cloud SQL + Firestore + Cloud Storage → Pub/Sub events → Cloud Monitoring / Logging / Security Command Center → Cloud Build CI/CD ← GitHub

Implementation Stages

1. **Stage 1 — Governance:** Folder, project, label, budget, IAM (Lab 1–2, Assignment 2)।
2. **Stage 2 — Networking:** VPC, firewall, NAT, PSC (Lab 4–6, Assignment 1)।
3. **Stage 3 — Platform:** GKE cluster, namespace, Workload Identity, একটি microservice deploy (Lab 18, Assignment 4)।
4. **Stage 4 — Data:** Cloud SQL, Firestore, Storage — প্রতিটি private ও backup সহ (Lab 11, 13–14)।
5. **Stage 5 — Application:** MERN অ্যাপ deploy, API Gateway, Pub/Sub event (Lab 15–16, 19, 24)।
6. **Stage 6 — CI/CD:** Cloud Build trigger, staging → production (Lab 21, Assignment 7)।
7. **Stage 7 — Observability & Security:** Dashboard, alert, audit, SCC review (Lab 22–23, Assignment 8–9)।

8. **Stage 8 — DR & FinOps:** Backup test, DR plan, cost report (Assignment 5–6)।

Verification

- প্রতিটি stage-এর verification ধাপ (Lab 1–24) সম্পন্ন হয়েছে কি না checklist দিয়ে যাচাই করুন।
- End-to-end: ব্যবহারকারী থেকে database পর্যন্ত পুরো পথ পরীক্ষা করুন।
- একটি pod ও একটি VM বন্ধ করে self-healing পরীক্ষা করুন।
- Backup থেকে restore পরীক্ষা করুন।

Security Review

- সম্পূর্ণ IAM audit — কোনো broad role নেই কি না।
- Firewall audit — SSH শুধু নিজের IP থেকে।
- SCC finding পর্যালোচনা ও remediation।
- Secret Manager-এ সব secret আছে কি না।

Cost Review

- পুরো platform-এর মাসিক খরচের অনুমান (Pricing Calculator)।
- Idle resource ও সাশ্রয়ের সুপারিশ।
- Budget alert যাচাই।

Final Deliverables

- চালু থাকা platform (বা Terraform code + deploy নথি)।
- Architecture diagram, network নকশা, IAM policy নথি।
- DR plan, runbook, security review, FinOps রিপোর্ট।
- CI/CD pipeline নথি ও monitoring dashboard।
- Cleanup পরিকল্পনা — প্রজেক্ট শেষে কী মুছবেন।

Capstone • Lab Workbook

Capstone — TechNova — Production-Ready Google Cloud Platform

এই capstone-এ আপনি স্বাধীনভাবে TechNova-র জন্য একটি production-ধরনের MERN platform ডিজাইন, deploy, সুরক্ষিত, monitor, troubleshoot ও document করবেন। আগের ল্যাব, assignment ও প্রজেক্টে যা শিখেছেন, সব এখানে একত্র হবে। সাহায্য ছাড়া নিজে করুন — তারপর Capstone Solution বিভাগের সঙ্গে মিলিয়ে দেখুন।

Scenario

TechNova একটি ছোট software কোম্পানি। তাদের MERN অ্যাপ (React frontend, Node.js/Express backend, PostgreSQL database, file upload) এখন GCP-তে production-এ চালাতে হবে। ব্যবহারকারী বাংলাদেশ ও দক্ষিণ এশিয়া থেকে আসবে। দল ছোট — মাত্র ৪ জন। বাজেট সীমিত, তবে নিরাপত্তা ও নির্ভরযোগ্যতায় আপস করা যাবে না। আপনার কাজ: সম্পূর্ণ platform তৈরি করা, যাতে পরের দলটি তা চালাতে পারে।

Requirements

- Production-ধরনের MERN অ্যাপ — frontend, backend, database, storage সব চালু থাকতে হবে।
- Secure networking — custom VPC, private database, least-privilege firewall।
- IAM — least privilege, service account, কোনো broad role নয়।
- Infrastructure as Code — Terraform দিয়ে মূল infrastructure।
- CI/CD — Git push থেকে deploy পর্যন্ত।
- Observability — dashboard, alert, uptime check, log।
- Security — Secret Manager, HTTPS, audit।
- Backup ও DR — backup, restore পরীক্ষা, RPO/RTO।
- Cost — Pricing Calculator অনুমান, budget alert।
- Documentation — architecture, runbook, rollback পরিকল্পনা।

Architecture Diagram

একটি architecture diagram আঁকুন যাতে দেখা যায়: ব্যবহারকারী → DNS → Load Balancer → Frontend/Backend (Cloud Run বা GKE) → Database (Cloud SQL) + Storage → Monitoring। প্রতিটি সংযোগে protocol ও port লিখুন।

User → Cloud DNS technova.example → HTTPS Load Balancer → Cloud Run technova-web + Cloud Run technova-api (or GKE) → Cloud SQL PostgreSQL (private IP) + Cloud Storage + Firestore → Pub/Sub → Cloud Monitoring / Logging / Alerts | Secret Manager | Cloud Build CI/CD

Project Hierarchy

- Organization (যদি থাকে) → Folder (`production`, `staging`) → Project (`technova-prod`, `technova-staging`)।
- প্রতিটি প্রজেক্টের নামকরণ, label (`env`, `team`, `cost-center`) ও billing budget লিখুন।
- কোন প্রজেক্টে কোন resource থাকবে তা টেবিলে লিখুন।

IAM Design

- ৪ জনের দলের জন্য role টেবিল লিখুন (Assignment 2-এর ধারণা ব্যবহার করুন)।
- Service account-গুলোর তালিকা ও প্রতিটির role লিখুন।
- CI/CD service account-এর role লিখুন।

VPC and Networking

- VPC, subnet (CIDR সহ), firewall rule, NAT, Private Google Access ও PSC-এর নকশা লিখুন।
- Database subnet-এ public IP নেই তা নিশ্চিত করুন।

Infrastructure as Code

- Terraform দিয়ে VPC, firewall, Cloud SQL, Cloud Run ও Storage-এর code লিখুন।
- Remote state (GCS backend) ও variable ব্যবহার করুন।
- `terraform plan` ও `apply`-এর ধাপ লিখুন।

Application Deployment

- Frontend ও backend-এর Dockerfile, image tag কৌশল ও Cloud Run/GKE deploy ধাপ লিখুন।
- Environment variable ও Secret Manager-এর ব্যবহার লিখুন।
- Health check ও readiness যাচাই লিখুন।

Database and Storage

- Cloud SQL-এর configuration: version, tier, storage, backup, private IP।
- Cloud Storage bucket-এর configuration: access, versioning, lifecycle।
- Firestore-এর ব্যবহার (যদি থাকে) লিখুন।

CI/CD

- Pipeline ধাপ: test → build → scan → staging → approval → production।
- `cloudbuild.yaml` বা GitHub Actions workflow লিখুন।
- Rollback ধাপ লিখুন।

Observability

- Dashboard, alert policy (কমপক্ষে ৩টি), uptime check ও log-based metric লিখুন।
- প্রতিটি alert-এর runbook ধাপ লিখুন।

Security Review

- IAM audit, firewall audit, SCC finding পর্যালোচনা করুন।
- Secret, HTTPS, public access — প্রতিটি যাচাই করুন।
- একটি security checklist তৈরি করুন।

Cost Estimate

- Pricing Calculator দিয়ে মাসিক অনুমান তৈরি করুন (কোনো price hardcode করবেন না)।
- সবচেয়ে বেশি খরচের তিনটি উৎস ও সাশ্রয়ের উপায় লিখুন।
- Billing budget ও alert সেট করুন।

Backup and Recovery

- Cloud SQL backup, point-in-time recovery ও restore পরীক্ষার ধাপ লিখুন।
- RPO ও RTO নির্ধারণ করুন।
- Storage-এর versioning ও lifecycle লিখুন।

Rollback Strategy

- Cloud Run revision rollback, GKE rollout undo ও database migration rollback — তিনটির ধাপ লিখুন।
- Rollback-এর সিদ্ধান্ত কে নেবে ও কখন নেবে তা লিখুন।

Operational Runbooks

- কমপক্ষে তিনটি incident-এর runbook লিখুন (Assignment 15-এর ধারণা ব্যবহার করুন)।
- প্রতিটি runbook-এ: লক্ষণ, যাচাই, কারণ, সমাধান, escalation, rollback।

Grading Rubric

ক্ষেত্র	ওজন	উত্তীর্ণের মানদণ্ড
Architecture ও networking	20%	VPC, subnet, firewall সঠিক; database private; diagram স্পষ্ট
IAM ও security	20%	Least privilege; secret Secret Manager-এ; HTTPS; audit সম্পন্ন
Application deployment	20%	Frontend, backend, database, storage — সব end-to-end কাজ করছে
IaC ও CI/CD	15%	Terraform code কাজ করছে; pipeline deploy করছে; rollback আছে

Observability ও DR	15%	Dashboard, alert, uptime check; backup restore পরীক্ষা করা হয়েছে
Documentation ও cost	10%	Runbook, architecture নথি, cost estimate — সব সম্পূর্ণ

সময়: ২–৩ সপ্তাহ (নিজের গতিতে)। **Cleanup:** Capstone শেষে সব billable resource মুছে ফেলুন, অথবা portfolio হিসেবে রাখলে billing budget যাচাই করুন ও খরচের নোট রাখুন।

Capstone Solution • Lab Workbook

Capstone Solution — Reference Solution Outline

এই বিভাগটি capstone-এর একটি **reference solution outline** — সম্পূর্ণ উত্তর নয়। প্রথমে নিজে capstone সম্পন্ন করুন, তারপর এই রূপরেখার সঙ্গে মিলিয়ে দেখুন কোথায় ঘাটতি আছে। এই রূপরেখার কোনো অংশই copy-paste করে জমা দেওয়ার জন্য নয়; এটি যাচাই করার জন্য।

> **সতর্কতা:** নিচের মানগুলো উদাহরণ মাত্র — IP, CIDR, নাম ও tier নিজের প্রজেক্ট অনুযায়ী বদলাতে হবে। কোনো price এখানে দেওয়া হয়নি; সর্বদা Google Cloud Pricing Calculator দেখুন।

1. Architecture (Reference)

User → Cloud DNS technova.example → Global External Application Load Balancer (HTTPS, Google-managed certificate) → Serverless NEG → Cloud Run technova-web (React/Next.js frontend) + Cloud Run technova-api (Node.js/Express backend, min 0, max 5) → Serverless VPC Access connector → Cloud SQL PostgreSQL (private IP 10.20.0.5, db-f1-micro / সবচেয়ে ছোট tier, 10 GB SSD, automated backup 02:00, PITR On) + Cloud Storage bucket technova-prod-uploads (uniform, public access prevention) + Firestore (tasks) → Pub/Sub technova-orders → Cloud Run functions (order processing) | Secret Manager (DB password) | Cloud Build CI/CD ← GitHub main | Cloud Monitoring dashboard + uptime check /health + alert (5xx > 5% for 5 min)

বিকল্প: GKE Autopilot cluster (`technova-gke`, `asia-south1`) — Deployment replica 2, HPA (CPU 60%), Ingress, Workload Identity। Cloud Run বা GKE — যেকোনো একটিই যথেষ্ট; দুটো একসঙ্গে করার দরকার নেই।

2. Project Hierarchy (Reference)

Level	নাম	উদ্দেশ্য
Organization	technova.example	সব প্রজেক্টের অভিভাবক
Folder	production / staging	Environment আলাদা করা
Project	technova-prod / technova-staging	Resource আলাদা করা
Labels	env=prod, team=platform, cost-center=technova	খরচ ও পরিচালনা

3. IAM Design (Reference)

Principal	Role	যুক্তি
Tech Lead	Project IAM Admin + Cloud Run Admin + Cloud SQL Admin	পরিচালনা, কিন্তু Owner নয়
Developer	Cloud Run Developer + Storage Object Admin (শুধু app bucket) + Cloud SQL Client	Deploy ও ডেটা ব্যবহার, IAM পরিবর্তন নয়

QA	Cloud Run Viewer + Monitoring Viewer	পরীক্ষা ও পর্যবেক্ষণ, পরিবর্তন নয়
Finance	Billing Account Viewer	শুধু খরচ দেখা
technova-app-sa	Cloud SQL Client + Storage Object Viewer/Creator + Secret Manager Secret Accessor	অ্যাপের প্রয়োজনীয় permission মাত্র
Cloud Build SA	Cloud Run Admin + Artifact Registry Writer	CI/CD deploy-এর জন্য

4. VPC and Networking (Reference)

- VPC: `technova-vpc`, custom subnet mode, region `asia-south1`।
- Subnet: web `10.10.1.0/24`, app `10.10.3.0/24`, db `10.10.2.0/24` — সব Private Google Access On।
- Firewall: SSH শুধু নিজের IP (`203.0.113.10/32` উদাহরণ) থেকে, HTTP/HTTPS শুধু web tag-এ, internal `10.10.0.0/16`।
- Cloud NAT (`technova-nat`) + Cloud Router (`technova-router`) — private VM-এর outbound।
- Private Service Access range `10.20.0.0/16` — Cloud SQL private IP-র জন্য।
- Serverless VPC Access connector `10.40.0.0/28` — Cloud Run থেকে private SQL-এ।

5. Infrastructure as Code (Reference)

Terraform module-এ VPC, subnet, firewall, NAT, Cloud SQL, Storage bucket ও Cloud Run service — প্রতিটি resource-এর code Lab 20-এর ধাঁচে লিখুন। Remote state: GCS bucket `technova-tf-state-CHANGE-ME-UNIQUE`, prefix `terraform/state`। Variable: `project_id`, `region`, `environment`। Secret variable-এ রাখবেন না।

6. Application Deployment (Reference)

- Backend: Node.js 20, Express, Dockerfile (multi-stage, non-root), image `asia-south1-docker.pkg.dev/technova-lab/technova-images/technova-api:v2` (digest দিয়ে deploy), Cloud Run min 0, max 5, concurrency 80, timeout 300s।
- Frontend: React/Next.js build, Cloud Run `technova-web`, API URL environment variable-এ।
- Health: `/health` — database সংযোগ যাচাই করে।
- Secret: `DB_PASSWORD` Secret Manager থেকে (`technova-db-password:latest`)।

7. Database and Storage (Reference)

- Cloud SQL: PostgreSQL 15, `db-f1-micro` (ল্যাবের জন্য), 10 GB SSD, private IP, automated backup 02:00, PITR On, retention 7 দিন, deletion protection production-এ On।
- Storage: bucket `technova-prod-uploads-CHANGE-ME-UNIQUE`, Standard, uniform access, public access prevention Enforced, versioning On, lifecycle: 30 দিন পর Nearline, 90 দিন পর Delete।
- Firestore: Native mode, `asia-south1`, Security Rules — শুধু প্রয়োজনীয় collection-এ read/write।

8. CI/CD (Reference)

Cloud Build trigger: GitHub `technova-api` repository, branch `main`, `cloudbuild.yaml` — test → build → push (tag `$SHORT_SHA`) → deploy to staging → manual approval → deploy to production। GitHub Actions বিকল্প: Workload Identity Federation দিয়ে authenticate, `deploy-cloudrun` action। Rollback: Cloud Run-এ আগের revision-এ traffic 100% ফিরিয়ে দিন।

9. Observability (Reference)

- Dashboard: request count, latency p95, error rate (5xx), CPU, database connection — পাঁচটি chart।
- Alert 1: 5xx error rate > 5% for 5 min → email। Alert 2: latency p95 > 1000 ms for 10 min → email। Alert 3: uptime check `/health` fails for 2 min → email।
- Uptime check: `/health`, প্রতি 1 মিনিট।
- Log-based metric: `severity>=ERROR` Cloud Run revision-এ।

10. Security Review (Reference Checklist)

- ☐ কোনো principal-কে Owner/Editor দেওয়া হয়নি।
- ☐ SSH শুধু নিজের IP থেকে; `0.0.0.0/0` থেকে SSH নেই।
- ☐ Database private IP-তে; public IP নেই।
- ☐ সব secret Secret Manager-এ; code-এ কোনো secret নেই।
- ☐ HTTPS বাধ্যতামূলক; HTTP redirect হচ্ছে।
- ☐ Bucket public নয়; uniform access On।
- ☐ Service account key তৈরি করা হয়নি।
- ☐ SCC finding পর্যালোচনা করা হয়েছে।

11. Cost Estimate (Reference Approach)

Google Cloud Pricing Calculator-এ যান। প্রতিটি service-এর অনুমান যোগ করুন: Cloud Run (request সংখ্যা ও memory), Cloud SQL (tier + storage + backup), Cloud Storage (GB + operation), Load Balancer (forwarding rule + data), NAT, GKE (যদি থাকে)। মোট মাসিক অনুমান লিখুন ও সবচেয়ে বেশি খরচের তিনটি উৎস চিহ্নিত করুন। Billing budget: মাসিক সীমা + alert 50% / 90% / 100%।

12. Backup and Recovery (Reference)

- RPO: 1 ঘণ্টা (PITR-এর কারণে), RTO: 2 ঘণ্টা (restore + verify)।
- Automated backup দৈনিক 02:00, retention 7 দিন।
- Restore পরীক্ষা: মাসে একবার clone তৈরি করে ডেটা যাচাই করুন।
- Storage: versioning On — ভুল delete হলে version থেকে ফেরত পাওয়া যায়।

13. Rollback Strategy (Reference)

- Cloud Run: `gcloud run services update-traffic --to-revisions=PREVIOUS-REVISION=100` ।
- GKE: `kubectl rollout undo deployment/technova-api` ।
- Database migration: প্রতিটি migration-এর সঙ্গে down migration লিখুন; migration-এর আগে backup নিন।
- সিদ্ধান্ত: Tech Lead rollback-এর সিদ্ধান্ত নেবেন; 5xx > 10% for 5 min হলে স্বয়ংক্রিয়ভাবে rollback বিবেচনা করুন।

14. Operational Runbooks (Reference — সংক্ষেপ)

- **API 5xx বেড়ে গেছে:** Dashboard দেখুন → Log Explorer-এ error খুঁজুন → Revision যাচাই করুন → প্রয়োজনে আগের revision-এ rollback করুন → Incident নোট লিখুন।
- **Database connection শেষ:** Cloud SQL-এর connection সংখ্যা দেখুন → অ্যাপের connection pool যাচাই করুন → প্রয়োজনে instance restart বা tier বাড়ান → Connection leak খুঁজুন।
- **খরচ হঠাৎ বেড়েছে:** Billing report দেখুন → কোন service বেড়েছে তা চিহ্নিত করুন → Min/max instances যাচাই করুন → প্রয়োজনে traffic সীমিত করুন।

15. Cleanup (Reference)

Capstone শেষে: Load balancer → Cloud Run service → GKE cluster → Cloud SQL instance → VPC Connector → Storage bucket → Artifact Registry image → Cloud Build trigger → DNS zone — এই ক্রমে মুছুন। শেষে `gcloud compute instances list`, `gcloud sql instances list`, `gcloud run services list`, `gcloud container clusters list` দিয়ে নিশ্চিত করুন কোনো billable resource চালু নেই। Billing budget ও cost report দেখে শেষ খরচ নোট করুন।

এই রূপরেখার সঙ্গে নিজের সমাধান মিলিয়ে দেখুন: কোন ক্ষেত্রে আপনি এগিয়ে আছেন, কোন ক্ষেত্রে আরও কাজ দরকার? ঘাটতি থাকলে সংশ্লিষ্ট Lab, Assignment বা Project-এ ফিরে যান। শুভকামনা — আপনি এখন একটি production-ধরনের GCP platform ডিজাইন ও চালাতে পারেন।